

COMPLETE SOURCE CODE REFERENCE

NeuraMorphix Recruitment 2026

Complete Source Code & Architecture Reference Manual for Frontend (React + TypeScript) and Backend (Spring Boot + MySQL)

Project: NeuraMorphix **Author:** NeuraMorphix Dev Team **Date:** September 2026

Backend (Spring Boot & MySQL)

Spring Boot Main Application

[backend/src/main/java/com/neuramorphix/recruitment_backend/RecruitmentBackendApplication.java](#)

Main entry point for the Spring Boot recruitment backend application.

```
package com.neuramorphix.recruitment_backend;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RecruitmentBackendApplication {

    public static void main(String[] args) {
        SpringApplication.run(RecruitmentBackendApplication.class, args);
    }

}
```

Security & CORS Configuration

[backend/src/main/java/com/neuramorphix/recruitment_backend/config/SecurityConfig.java](#)

Spring Security filter chain configuring CORS allow-all headers and open public API endpoints.

```
package com.neuramorphix.recruitment_backend.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.List;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .anyRequest().permitAll());

        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOriginPatterns(List.of("*"));
        config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH"));
        config.setAllowedHeaders(List.of("*"));
        config.setAllowCredentials(true);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }

}
```

User Authentication Controller

[backend/src/main/java/com/neuramorphix/recruitment_backend/controller/UserController.java](#)

REST controller handling admin/recruiter registration and login (/api/users/login, /api/users/register).

```
package com.neuramorphix.recruitment_backend.controller;

import com.neuramorphix.recruitment_backend.entity.User;
import com.neuramorphix.recruitment_backend.repository.UserRepository;
import com.neuramorphix.recruitment_backend.service.UserService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserRepository userRepository;
    private final UserService userService;

    public UserController(UserRepository userRepository, UserService userService) {
        this.userRepository = userRepository;
        this.userService = userService;
    }

    @PostMapping("/register")
    public User registerUser(@RequestBody User user) {
        return userRepository.save(user);
    }

    @PostMapping("/login")
    public User login(@RequestBody User user) {

        User loggedInUser = userService.login(
            user.getEmail(),
            user.getPassword()
        );

        return loggedInUser;
    }
}
```

Applicant Controller

[backend/src/main/java/com/neuramorphix/recruitment_backend/controller/ApplicantController.java](#)

REST controller handling candidate application submissions, retrieving applicants, and updating statuses (/api/applicants).

```
package com.neuramorphix.recruitment_backend.controller;

import com.neuramorphix.recruitment_backend.entity.ApplicantEntity;
import com.neuramorphix.recruitment_backend.repository.ApplicantRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/applicants")
@CrossOrigin(origins = "*")
public class ApplicantController {

    private final ApplicantRepository applicantRepository;

    public ApplicantController(ApplicantRepository applicantRepository) {
        this.applicantRepository = applicantRepository;
    }

    @GetMapping
    public List<ApplicantEntity> getAllApplicants() {
```

```

        return applicantRepository.findAll();
    }

    @GetMapping("/{appId}")
    public ResponseEntity<ApplicantEntity> getApplicantById(@PathVariable String appId) {
        Optional<ApplicantEntity> applicant = applicantRepository.findById(appId);
        return applicant.map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<ApplicantEntity> createApplicant(@RequestBody ApplicantEntity applicant) {
        if (applicant.getCreatedAt() == null) {
            applicant.setCreatedAt(LocalDateTime.now());
        }
        applicant.setUpdatedAt(LocalDateTime.now());
        ApplicantEntity saved = applicantRepository.save(applicant);
        return ResponseEntity.ok(saved);
    }

    @PutMapping("/{appId}/status")
    public ResponseEntity<ApplicantEntity> updateStatus(
        @PathVariable String appId,
        @RequestBody ApplicantEntity updates) {
        Optional<ApplicantEntity> existingOpt = applicantRepository.findById(appId);
        if (existingOpt.isEmpty()) {
            return ResponseEntity.notFound().build();
        }

        ApplicantEntity existing = existingOpt.get();
        if (updates.getStatus() != null) existing.setStatus(updates.getStatus());
        if (updates.getFinalAssignedTeam() != null) existing.setFinalAssignedTeam(updates.getFinalAssignedTeam());
        if (updates.getDeclineReason() != null) existing.setDeclineReason(updates.getDeclineReason());
        if (updates.getRequestedInfoQuestion() != null) existing.setRequestedInfoQuestion(updates.getRequestedInfoQuestion());
        if (updates.getRequestedInfoResponse() != null) existing.setRequestedInfoResponse(updates.getRequestedInfoResponse());
        if (updates.getInterviewDetails() != null) existing.setInterviewDetails(updates.getInterviewDetails());
        existing.setUpdatedAt(LocalDateTime.now());

        ApplicantEntity saved = applicantRepository.save(existing);
        return ResponseEntity.ok(saved);
    }
}

```

User JPA Entity

[backend/src/main/java/com/neuramorphix/recruitment_backend/entity/User.java](#)

Database entity mapping to the MySQL "users" table.

```

package com.neuramorphix.recruitment_backend.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(nullable = false)
    private String password;

    private String role;

    public User() {
    }
}

```

```

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }

    public void setRole(String role) {
        this.role = role;
    }
}

```

Applicant JPA Entity

[backend/src/main/java/com/neuramorphix/recruitment_backend/entity/ApplicantEntity.java](#)

Database entity mapping to the MySQL "applicants" table.

```

package com.neuramorphix.recruitment_backend.entity;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "applicants")
public class ApplicantEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "application_id", unique = true, nullable = false)
    private String applicationId;

    @Column(name = "full_name", nullable = false)
    private String fullName;

    @Column(nullable = false)
    private String email;

    @Column(nullable = false)
    private String phone;

    private String college;
    private String department;
}

```

```

@Column(name = "year_of_study")
private String yearOfStudy;

@Column(name = "first_preference", nullable = false)
private String firstPreference;

@Column(name = "second_preference")
private String secondPreference;

@Column(columnDefinition = "TEXT")
private String skills;

@Column(name = "experience_summary", columnDefinition = "TEXT")
private String experienceSummary;

@Column(name = "why_join", columnDefinition = "TEXT")
private String whyJoin;

@Column(name = "github_url")
private String githubUrl;

@Column(name = "linkedin_url")
private String linkedinUrl;

@Column(name = "portfolio_url")
private String portfolioUrl;

@Column(nullable = false)
private String status = "Applied";

@Column(name = "final_assigned_team")
private String finalAssignedTeam;

@Column(name = "decline_reason")
private String declineReason;

@Column(name = "requested_info_question", columnDefinition = "TEXT")
private String requestedInfoQuestion;

@Column(name = "requested_info_response", columnDefinition = "TEXT")
private String requestedInfoResponse;

@Column(name = "interview_details", columnDefinition = "TEXT")
private String interviewDetails;

@Column(name = "created_at")
private LocalDateTime createdAt = LocalDateTime.now();

@Column(name = "updated_at")
private LocalDateTime updatedAt = LocalDateTime.now();

public ApplicantEntity() {
}

// Getters and Setters
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }

public String getApplicationId() { return applicationId; }
public void setApplicationId(String applicationId) { this.applicationId = applicationId; }

public String getFullName() { return fullName; }
public void setFullName(String fullName) { this.fullName = fullName; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }

public String getCollege() { return college; }
public void setCollege(String college) { this.college = college; }

public String getDepartment() { return department; }
public void setDepartment(String department) { this.department = department; }

```

```

public String getYearOfStudy() { return yearOfStudy; }
public void setYearOfStudy(String yearOfStudy) { this.yearOfStudy = yearOfStudy; }

public String getFirstPreference() { return firstPreference; }
public void setFirstPreference(String firstPreference) { this.firstPreference = firstPreference; }

public String getSecondPreference() { return secondPreference; }
public void setSecondPreference(String secondPreference) { this.secondPreference = secondPreference; }

public String getSkills() { return skills; }
public void setSkills(String skills) { this.skills = skills; }

public String getExperienceSummary() { return experienceSummary; }
public void setExperienceSummary(String experienceSummary) { this.experienceSummary = experienceSummary; }

public String getWhyJoin() { return whyJoin; }
public void setWhyJoin(String whyJoin) { this.whyJoin = whyJoin; }

public String getGithubUrl() { return githubUrl; }
public void setGithubUrl(String githubUrl) { this.githubUrl = githubUrl; }

public String getLinkedinUrl() { return linkedinUrl; }
public void setLinkedinUrl(String linkedinUrl) { this.linkedinUrl = linkedinUrl; }

public String getPortfolioUrl() { return portfolioUrl; }
public void setPortfolioUrl(String portfolioUrl) { this.portfolioUrl = portfolioUrl; }

public String getStatus() { return status; }
public void setStatus(String status) { this.status = status; }

public String getFinalAssignedTeam() { return finalAssignedTeam; }
public void setFinalAssignedTeam(String finalAssignedTeam) { this.finalAssignedTeam = finalAssignedTeam; }

public String getDeclineReason() { return declineReason; }
public void setDeclineReason(String declineReason) { this.declineReason = declineReason; }

public String getRequestedInfoQuestion() { return requestedInfoQuestion; }
public void setRequestedInfoQuestion(String requestedInfoQuestion) { this.requestedInfoQuestion = requestedInfoQuestion; }

public String getRequestedInfoResponse() { return requestedInfoResponse; }
public void setRequestedInfoResponse(String requestedInfoResponse) { this.requestedInfoResponse = requestedInfoResponse; }

public String getInterviewDetails() { return interviewDetails; }
public void setInterviewDetails(String interviewDetails) { this.interviewDetails = interviewDetails; }

public LocalDateTime getCreatedAt() { return createdAt; }
public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

public LocalDateTime getUpdatedAt() { return updatedAt; }
public void setUpdatedAt(LocalDateTime updatedAt) { this.updatedAt = updatedAt; }
}

```

User Repository

[backend/src/main/java/com/neuramorphix/recruitment_backend/repository/UserRepository.java](#)

Spring Data JPA repository for user lookup by email.

```

package com.neuramorphix.recruitment_backend.repository;

import com.neuramorphix.recruitment_backend.entity.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {

    User findByEmail(String email);

}

```

Applicant Repository

[backend/src/main/java/com/neuramorphix/recruitment_backend/repository/ApplicantRepository.java](#)

Spring Data JPA repository for applicant lookup by application ID and email.

```
package com.neuramorphix.recruitment_backend.repository;

import com.neuramorphix.recruitment_backend.entity.ApplicantEntity;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface ApplicantRepository extends JpaRepository<ApplicantEntity, Long> {
    Optional<ApplicantEntity> findById(String applicationId);
    Optional<ApplicantEntity> findByEmail(String email);
    Optional<ApplicantEntity> findByIdAndEmail(String applicationId, String email);
}
```

User Authentication Service

[backend/src/main/java/com/neuramorphix/recruitment_backend/service/UserService.java](#)

Business logic for verifying admin/recruiter passwords during login.

```
package com.neuramorphix.recruitment_backend.service;

import com.neuramorphix.recruitment_backend.entity.User;
import com.neuramorphix.recruitment_backend.repository.UserRepository;
import org.springframework.stereotype.Service;

@Service
public class UserService {

    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User login(String email, String password) {

        User user = userRepository.findByEmail(email);

        if (user != null && user.getPassword().equals(password)) {
            return user;
        }

        return null;
    }
}
```

Application Properties Configuration

[backend/src/main/resources/application.properties](#)

Database connection properties (MySQL URL, username, password, JPA Hibernate ddl-auto).

```
spring.application.name=recruitment-backend

spring.datasource.url=jdbc:mysql://localhost:3306/recruitment_db
spring.datasource.username=root
spring.datasource.password=50951522Mone

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```


Maven build configuration containing Spring Boot, Spring Security, JPA, and MySQL dependencies.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.neuramorphix</groupId>
  <artifactId>recruitment-backend</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name/>
  <description/>
  <url/>
  <licenses>
    <license/>
  </licenses>
  <developers>
    <developer/>
  </developers>
  <scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
  </scm>
  <properties>
    <java.version>21</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>

    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-webmvc-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
                <execution>
                    <id>default-testCompile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>testCompile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
</project>

```

Email & Serverless API

Vercel Serverless Email Function

api/send-email.js

Serverless Node.js endpoint using Nodemailer to send branded recruitment emails via Gmail SMTP.

```
import nodemailer from 'nodemailer';

function escapeHtml(value) {
  if (value === null || value === undefined) {
    return '';
  }

  return String(value)
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;');
}

function buildHtml({
  applicantName,
  applicationId,
  phone,
  firstPreference,
  secondPreference,
  finalAssignedTeam,
  requestedInfoQuestion,
  interviewDetails,
  emailType,
  bodyHtml,
}) {
  const safeName = escapeHtml(applicantName);
  const safeApplicationId = escapeHtml(applicationId);
  const safePhone = escapeHtml(phone);
  const safeFirstPreference = escapeHtml(firstPreference);
  const safeSecondPreference = escapeHtml(secondPreference);
  const safeFinalTeam = escapeHtml(finalAssignedTeam);
  const safeQuestion = escapeHtml(requestedInfoQuestion);
  const safeInterview = escapeHtml(interviewDetails);

  let additionalContent = '';

  if (emailType === 'application_received') {
    additionalContent = `
    <p>
      Your application has been successfully received by
      the NeuraMorphix recruitment team.
    </p>

    <div class="info-box">
      <strong>Application ID:</strong>
      ${safeApplicationId}
    </div>
  `;
  }

  if (emailType === 'shortlisted') {
    additionalContent = `
    <p>
      Congratulations! Your application has been shortlisted
      for the next stage of the recruitment process.
    </p>

    <div class="info-box">
      <strong>Application ID:</strong>
      ${safeApplicationId}
    </div>
  `;
  }
}
```

```

if (emailType === 'interview') {
  additionalContent = `
    <p>
      You have been selected for an interview.
    </p>

    <div class="info-box">
      <strong>Interview Details:</strong><br />
      ${safeInterview}
    </div>
  `;
}

if (emailType === 'info_requested') {
  additionalContent = `
    <p>
      Additional information is required for your application.
    </p>

    <div class="info-box">
      <strong>Requested Information:</strong><br />
      ${safeQuestion}
    </div>
  `;
}

if (emailType === 'accepted') {
  additionalContent = `
    <p>
      Congratulations! Your application has been accepted.
    </p>

    <div class="info-box">
      <strong>Assigned Team:</strong>
      ${safeFinalTeam}
    </div>
  `;
}

if (emailType === 'declined') {
  additionalContent = `
    <p>
      Thank you for your interest in NeuraMorphix.
      After reviewing your application, we are unable to
      proceed with your application at this time.
    </p>
  `;
}

return `
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" />
  <title>NeuraMorphix Recruitment</title>

  <style>
    body {
      margin: 0;
      padding: 0;
      background: #0f172a;
      font-family: Arial, Helvetica, sans-serif;
      color: #e2e8f0;
    }

    .container {
      max-width: 650px;
      margin: 40px auto;
      background: #111827;
      border-radius: 16px;
      overflow: hidden;
      border: 1px solid #334155;
    }

    .header {

```

```
padding: 28px;
background: #020617;
text-align: center;
}

.header h1 {
margin: 0;
color: #22d3ee;
font-size: 26px;
}

.content {
padding: 32px;
line-height: 1.7;
}

.info-box {
margin: 20px 0;
padding: 16px;
background: #0f172a;
border: 1px solid #155e75;
border-radius: 10px;
color: #67e8f9;
}

.details {
margin-top: 20px;
padding: 18px;
background: #020617;
border-radius: 10px;
}

.details p {
margin: 8px 0;
}

.footer {
padding: 20px;
background: #020617;
text-align: center;
font-size: 12px;
color: #94a3b8;
}

strong {
color: #f8fafc;
}
</style>
</head>

<body>
<div class="container">

<div class="header">
<h1>NeuraMorphix</h1>
<p>Recruitment 2026</p>
</div>

<div class="content">

<p>Hello <strong>${safeName}</strong></p>

${additionalContent}

<div class="details">
<p>
<strong>Application ID:</strong>
${safeApplicationId}
</p>

<p>
<strong>Phone:</strong>
${safePhone}
</p>

<p>
```

```

        <strong>First Preference:</strong>
        ${safeFirstPreference}
    </p>

    <p>
        <strong>Second Preference:</strong>
        ${safeSecondPreference}
    </p>
</div>

${
    bodyHtml
    ? `<div style="margin-top:20px">${bodyHtml}</div>`
    : ''
}

<p>
    Thank you for your interest in NeuraMorphix.
</p>

<p>
    Regards,<br />
    <strong>NeuraMorphix Recruitment Team</strong>
</p>

</div>

<div class="footer">
    This is an automated recruitment notification.
</div>

</div>
</body>
</html>
`;
}

```

```

export default async function handler(req, res) {
    // CORS
    res.setHeader('Access-Control-Allow-Origin', '*');
    res.setHeader(
        'Access-Control-Allow-Methods',
        'POST, OPTIONS'
    );
    res.setHeader(
        'Access-Control-Allow-Headers',
        'Content-Type'
    );

    // OPTIONS
    if (req.method === 'OPTIONS') {
        return res.status(200).end();
    }

    // Only POST
    if (req.method !== 'POST') {
        return res.status(405).json({
            success: false,
            message: 'Method not allowed',
        });
    }

    try {
        const {
            to,
            subject,
            bodyHtml,
            applicantName,
            applicationId,
            phone,
            firstPreference,
            secondPreference,
            finalAssignedTeam,
            requestedInfoQuestion,
            interviewDetails,
            emailType,
        } = req.body;
    } catch (error) {
        return res.status(400).json({
            success: false,
            message: 'Invalid request body',
        });
    }

    // ... (rest of the handler logic)
}

```

```

} = req.body || {};

// Required fields
if (!to || !subject || !applicantName || !applicationId) {
  return res.status(400).json({
    success: false,
    message:
      'Missing required email information',
  });
}

// Environment variables
const systemEmail = process.env.SMTP_USER || 'moniswarmoni509@gmail.com';
const systemPass = process.env.SMTP_PASS || 'rzlcebjxhpgbumqb';

// Gmail SMTP
const transporter = nodemailer.createTransport({
  service: 'gmail',

  auth: {
    user: systemEmail,
    pass: systemPass,
  },
});

// Build email
const html = buildHtml({
  applicantName,
  applicationId,
  phone,
  firstPreference,
  secondPreference,
  finalAssignedTeam,
  requestedInfoQuestion,
  interviewDetails,
  emailType,
  bodyHtml,
});

// Send
const info = await transporter.sendMail({
  from: `"NeuraMorphix Recruitment" <${systemEmail}>`,
  to,
  subject,
  html,
});

console.log(
  'Email successfully sent:',
  info.messageId
);

return res.status(200).json({
  success: true,
  message: 'Email sent successfully',
  messageId: info.messageId,
  via: 'Gmail SMTP',
});

} catch (error) {
  console.error(
    'Nodemailer error:',
    error
  );

  return res.status(500).json({
    success: false,
    message:
      error instanceof Error
        ? error.message
        : 'Failed to send email',
  });
}
}

```

Vite build config with Tailwind CSS, local network exposure (--host), and dev /api/send-email middleware.

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';
// @ts-ignore
import nodemailer from 'nodemailer';
// @ts-ignore
import { readFileSync } from 'fs';
// @ts-ignore
import { resolve } from 'path';

// Load .env manually for Vite plugins (they run in Node context)
function loadDotEnv(): Record<string, string> {
  try {
    const envPath = resolve(process.cwd(), '.env');
    const content = readFileSync(envPath, 'utf-8');
    const vars: Record<string, string> = {};
    content.split('\n').forEach((line: string) => {
      const trimmed = line.trim();
      if (!trimmed || trimmed.startsWith('#')) return;
      const eqIdx = trimmed.indexOf('=');
      if (eqIdx !== -1) {
        const key = trimmed.slice(0, eqIdx).trim();
        const value = trimmed.slice(eqIdx + 1).trim();
        vars[key] = value;
      }
    });
    return vars;
  } catch {
    return {};
  }
}

function buildHtml({ applicantName, applicationId, phone, firstPreference, secondPreference, emailType }: any) {
  const typeLabel: Record<string, string> = {
    application_received: 'Application Received',
    shortlisted: 'Shortlisted',
    interview: 'Interview Scheduled',
    info_requested: 'Additional Information Requested',
    accepted: 'Application Accepted',
    declined: 'Application Update',
  };
  const label = typeLabel[emailType] || 'Recruitment Update';

  return `<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>NeuraMorphix Recruitment</title>
</head>
<body style="margin:0;padding:0;background:#0f172a;font-family:Arial,sans-serif;">
  <table width="100%" cellpadding="0" cellspacing="0" style="background:#0f172a;padding:32px 16px;">
    <tr>
      <td align="center">
        <table width="100%" style="max-width:560px;background:#1e293b;border-radius:16px;overflow:hidden;border:1px solid #334155;">
          <tr>
            <td style="background:linear-gradient(135deg,#0e7490,#1d4ed8);padding:32px 24px;text-align:center;">
              <p style="margin:0 0 6px 0;color:#bae6fd;font-size:11px;font-weight:700;letter-spacing:2px;text-transform:uppercase;">NeuraMorphix · Recruitment 2026</p>
              <h1 style="margin:0;color:#ffffff;font-size:24px;font-weight:900;letter-spacing:-0.5px;">NeuraMorphix Recruitment</h1>
              <p style="margin:10px 0 0 0;color:#bae6fd;font-size:13px;">${label}</p>
            </td>
          </tr>
          <tr>
            <td style="padding:32px 24px;">
              <p style="margin:0 0 16px 0;color:#e2e8f0;font-size:16px;font-weight:600;">Hello ${applicantName || 'Applicant'},
              <p style="margin:0 0 20px 0;color:#94a3b8;font-size:14px;line-height:1.7;">
```



```

        Welcome to <strong style="color:#38bdf8;">NeuraMorphix</strong>! We are thrilled to receive your application for
the
        <strong style="color:#f8fafc;">NeuraMorphix 2026 Team Recruitment</strong>. Your application has been
successfully
        registered in our system and is now under review by our recruitment team.
    </p>
    <table width="100%" cellpadding="0" cellspacing="0" style="background:#0f172a;border-radius:12px;border:1px solid
#334155;margin-bottom:24px;">
        <tr>
            <td style="padding:20px 24px;">
                <p style="margin:0 0 14px 0;color:#38bdf8;font-size:11px;font-weight:700;letter-spacing:1.5px;text-
transform:uppercase;border-bottom:1px solid #1e293b;padding-bottom:10px;">Application Details</p>
                <table width="100%" cellpadding="0" cellspacing="0">
                    <tr>
                        <td style="padding:5px 0;color:#64748b;font-size:12px;font-weight:600;width:140px;">Application ID</td>
                        <td style="padding:5px 0;color:#38bdf8;font-size:14px;font-weight:900;font-family:monospace;letter-
spacing:1px;">${applicationId || 'N/A'}</td>
                    </tr>
                    <tr>
                        <td style="padding:5px 0;color:#64748b;font-size:12px;font-weight:600;">Phone Number</td>
                        <td style="padding:5px 0;color:#e2e8f0;font-size:13px;font-weight:600;">${phone || 'N/A'}</td>
                    </tr>
                    <tr>
                        <td style="padding:5px 0;color:#64748b;font-size:12px;font-weight:600;">1st Preference</td>
                        <td style="padding:5px 0;color:#e2e8f0;font-size:13px;font-weight:600;">${firstPreference || 'N/A'}</td>
                    </tr>
                    <tr>
                        <td style="padding:5px 0;color:#64748b;font-size:12px;font-weight:600;">2nd Preference</td>
                        <td style="padding:5px 0;color:#e2e8f0;font-size:13px;font-weight:600;">${secondPreference || 'None (Optional)'}</td>
                    </tr>
                    <tr>
                        <td style="padding:5px 0;color:#64748b;font-size:12px;font-weight:600;">2nd Preference</td>
                        <td style="padding:5px 0;color:#e2e8f0;font-size:13px;font-weight:600;">${secondPreference}</td>
                    </tr>
                </table>
                <td style="padding:5px 0;color:#64748b;font-size:12px;font-weight:600;">Status</td>
                <td style="padding:5px 0;"><span style="background:#064e3b;color:#6ee7b7;font-size:11px;font-
weight:700;padding:3px 10px;border-radius:20px;text-transform:uppercase;">${label}</span></td>
            </tr>
        </table>
    </td>
</tr>
</table>
<p style="margin:0 0 10px 0;color:#94a3b8;font-size:13px;line-height:1.7;">
    Please <strong style="color:#f8fafc;">save your Application ID</strong> – you will need it to track your
recruitment status on our portal at any time.
</p>
<p style="margin:0 0 24px 0;color:#94a3b8;font-size:13px;line-height:1.7;">
    Our recruitment team will review all applications and update your status accordingly. You will receive further
updates at this email address.
</p>
<p style="margin:0;color:#64748b;font-size:12px;border-top:1px solid #334155;padding-top:20px;line-height:1.7;">
    Thank you for applying and for your interest in joining NeuraMorphix.<br/>
    We look forward to reviewing your application!<br/><br/>
    <strong style="color:#38bdf8;">Thank you,<br/>The NeuraMorphix Team</strong><br/>
    <span style="color:#475569;">NeuraMorphix Recruitment System · moniswarmoni509@gmail.com</span>
</p>
</td>
</tr>
</table>
<td style="background:#0f172a;padding:16px 24px;text-align:center;border-top:1px solid #1e293b;">
    <p style="margin:0;color:#334155;font-size:11px;">© 2026 NeuraMorphix · All rights reserved</p>
</td>
</tr>
</table>
</td>
</tr>
</table>
</body>
</html>`;
}

```

```

function nodemailerPlugin() {
    const env = loadDotEnv();
    const systemEmail = env.SMTP_USER || process.env.SMTP_USER || 'moniswarmoni509@gmail.com';
    const systemPass = env.SMTP_PASS || process.env.SMTP_PASS || 'rzlcebjxhpgbumqb';

    const isRealPassword = systemPass.length > 0 && !systemPass.includes('your_gmail_app_password');
}

```

```

// Primary Gmail transporter
const gmailTransporter = nodemailer.createTransport({
  service: 'gmail',
  auth: { user: systemEmail, pass: systemPass },
  tls: { rejectUnauthorized: false },
});

let testTransporter: nodemailer.Transporter | null = null;
async function getEtherealTransporter() {
  if (!testTransporter) {
    console.log('\n[NeuraMorphix Mailer] Creating Ethereal test account...');
    const testAccount = await nodemailer.createTestAccount();
    testTransporter = nodemailer.createTransport({
      host: 'smtp.ethereal.email',
      port: 587,
      secure: false,
      auth: { user: testAccount.user, pass: testAccount.pass },
    });
    console.log('[NeuraMorphix Mailer] Ethereal test account ready:', testAccount.user);
  }
  return testTransporter;
}

if (!isRealPassword) {
  console.log('\n⚠ [NeuraMorphix Mailer] No Gmail App Password found in .env (SMTP_PASS).');
  console.log('  Emails will use Ethereal test preview (not delivered to real inbox).');
  console.log('  To send real emails: set SMTP_PASS=<16-char Gmail App Password> in .env\n');
} else {
  console.log('\n✅ [NeuraMorphix Mailer] Gmail configured for ${systemEmail}');
  console.log('  Real emails will be sent immediately via Gmail SMTP.\n');
}

return {
  name: 'vite-plugin-nodemailer',
  configureServer(server: any) {
    server.middlewares.use('/api/send-email', async (req: any, res: any) => {
      // CORS Headers for mobile devices connecting over WiFi / Network IP
      res.setHeader('Access-Control-Allow-Origin', '*');
      res.setHeader('Access-Control-Allow-Methods', 'GET, POST, OPTIONS');
      res.setHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization, X-Requested-With');

      if (req.method === 'OPTIONS') {
        res.statusCode = 200;
        res.end();
        return;
      }

      if (req.method !== 'POST') {
        res.statusCode = 405;
        res.end('Method Not Allowed');
        return;
      }

      let bodyStr = '';
      req.on('data', (chunk: any) => { bodyStr += chunk.toString(); });
      req.on('end', async () => {
        try {
          const body = JSON.parse(bodyStr || '{}');
          const { to, subject, applicantName, applicationId, phone, firstPreference, secondPreference, emailType } = body;

          const recipientEmail = to || systemEmail;
          const emailSubject = subject || `NeuraMorphix Recruitment – Application Received (${applicationId || 'N/A'})`;
          const htmlContent = buildHtml({ applicantName, applicationId, phone, firstPreference, secondPreference, emailType });

          const mailOptions = {
            from: `NeuraMorphix Recruitment <${systemEmail}>`,
            to: recipientEmail,
            subject: emailSubject,
            html: htmlContent,
          };

          let info: any;
          let sentVia = 'Gmail';

          if (isRealPassword) {
            try {

```

```

        info = await gmailTransporter.sendMail(mailOptions);
        console.log(`\n✅ [NeuraMorphix Mailer] Email sent to ${recipientEmail} via Gmail`);
        console.log(`    Message ID: ${info.messageId}`);
    } catch (gmailErr: any) {
        console.log(`\n⚠️ [NeuraMorphix Mailer] Gmail failed: ${gmailErr.message}`);
        console.log('    Falling back to Ethereum preview...');
        const ethereal = await getEtherealTransporter();
        info = await ethereal.sendMail(mailOptions);
        sentVia = 'Ethereal (preview)';
        const previewUrl = nodemailer.getTestMessageUrl(info);
        console.log(`\n🔗 [NeuraMorphix Mailer] Email preview: ${previewUrl}\n`);
    }
} else {
    // No real password – use Ethereum for preview
    const ethereal = await getEtherealTransporter();
    info = await ethereal.sendMail(mailOptions);
    sentVia = 'Ethereal (preview)';
    const previewUrl = nodemailer.getTestMessageUrl(info);
    console.log(`\n🔗 [NeuraMorphix Mailer] Email preview (open in browser): ${previewUrl}\n`);
}

res.setHeader('Content-Type', 'application/json');
res.statusCode = 200;
res.end(JSON.stringify({ success: true, messageId: info.messageId, via: sentVia }));
} catch (err: any) {
    console.error(`\n❌ [NeuraMorphix Mailer] Error:`, err.message);
    res.setHeader('Content-Type', 'application/json');
    res.statusCode = 500;
    res.end(JSON.stringify({ success: false, error: err.message }));
}
});
});
},
});
}

export default defineConfig({
  plugins: [react(), tailwindcss(), nodemailerPlugin()],
  server: {
    host: true, // Listen on all network addresses (0.0.0.0) for mobile access
    cors: true,
  },
  preview: {
    host: true,
    cors: true,
  },
});

```

Frontend Services & Types

TypeScript Recruitment Types

[src/types/recruitment.ts](#)

TypeScript interface definitions for Applicant, Role, EmailLog, AdminUser, and EmailSettings.

```
export type ApplicationStatus =
  | 'Application Received'
  | 'Under Review'
  | 'Shortlisted'
  | 'Interview'
  | 'Information Requested'
  | 'Information Received'
  | 'Accepted'
  | 'Declined';

export type DeclineReasonCategory =
  | 'Role capacity reached'
  | 'Skills mismatch'
  | 'Application incomplete'
  | 'Selection criteria'
  | 'Other';

export interface AdminNote {
  id: string;
  author: string;
  text: string;
  created_at: string;
}

export interface Applicant {
  id: string;
  application_id: string;
  full_name: string;
  email: string;
  phone: string;
  college: string;
  department: string;
  year: string;
  skills: string[];
  experience: string;
  first_preference: string;
  second_preference: string;
  final_assigned_team: string | null;
  status: ApplicationStatus;
  resume_url: string;
  github_url: string;
  linkedin_url: string;
  portfolio_url: string;
  admin_notes: AdminNote[];
  decline_reason: DeclineReasonCategory | null;
  decline_note: string | null;
  requested_info_question: string | null;
  requested_info_response: string | null;
  interview_details: string | null;
  created_at: string;
  updated_at: string;
  reviewed_at: string | null;
  accepted_at: string | null;
  declined_at: string | null;
  accepted_by?: string;
  declined_by?: string;
}

export interface Role {
  role_id: string;
  role_name: string;
  description: string;
  skills: string[];
  icon_name: string;
  is_active: boolean;
```

```

}

export type EmailType =
  | 'application_received'
  | 'shortlisted'
  | 'interview'
  | 'info_requested'
  | 'accepted'
  | 'declined';

export interface EmailLog {
  email_id: string;
  application_id: string;
  recipient_email: string;
  email_type: EmailType;
  subject: string;
  body_html: string;
  status: 'Sent' | 'Failed';
  sent_at: string;
}

export interface AdminUser {
  admin_id: string;
  name: string;
  email: string;
  role: 'Lead Recruiter' | 'Technical Reviewer' | 'Admin';
  password?: string;
  created_at: string;
}

export interface EmailTemplate {
  subject: string;
  body_template: string;
}

export interface EmailSettings {
  enable_application_received: boolean;
  enable_shortlist: boolean;
  enable_interview: boolean;
  enable_info_requested: boolean;
  enable_acceptance: boolean;
  enable_decline: boolean;
  templates: Record<EmailType, EmailTemplate>;
}

export interface RecruitmentConfig {
  start_date: string;
  end_date: string;
  is_manually_open: boolean | null; // null means auto by date
}

```

Backend API Integration Service

[src/services/api.ts](#)

Client API service bridging frontend components to Spring Boot backend (/api/users and /api/applicants).

```

import type { Applicant, AdminUser } from '../types/recruitment';
import { DatabaseService } from './db';

const BACKEND_BASE_URL = 'http://localhost:8080';

export class BackendApiService {
  /**
   * Attempt to authenticate via Spring Boot backend (/api/users/login)
   * Falls back to local DatabaseService if backend is offline.
   */
  static async loginUser(email: string, password: string): Promise<AdminUser | null> {
    try {
      const response = await fetch(`${BACKEND_BASE_URL}/api/users/login`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password }),
      });
    }

```

```

    if (response.ok) {
      const user = await response.json();
      if (user && user.email) {
        return {
          admin_id: `admin-${user.id} || 'db'`,
          name: user.name || user.email.split('@')[0],
          email: user.email,
          role: (user.role as any) || 'admin',
          created_at: new Date().toISOString(),
        };
      }
    }
  } catch {
    console.log('[BackendApiService] Backend offline, using local authentication.');
```

```

  }

  // Fallback to local authentication
  return DatabaseService.authenticateAdmin(email, password);
}

/**
 * Register a user via Spring Boot backend (/api/users/register)
 */
static async registerUser(name: string, email: string, password: string, role = 'admin'): Promise<boolean> {
  try {
    const response = await fetch(`${BACKEND_BASE_URL}/api/users/register`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name, email, password, role }),
    });
    return response.ok;
  } catch (err) {
    console.error('[BackendApiService] Register error:', err);
    return false;
  }
}

/**
 * Sync new applicant to Spring Boot backend (/api/applicants)
 */
static async syncApplicant(applicant: Applicant): Promise<void> {
  try {
    await fetch(`${BACKEND_BASE_URL}/api/applicants`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        applicationId: applicant.application_id,
        fullName: applicant.full_name,
        email: applicant.email,
        phone: applicant.phone,
        college: applicant.college,
        department: applicant.department,
        yearOfStudy: applicant.year,
        firstPreference: applicant.first_preference,
        secondPreference: applicant.second_preference,
        skills: applicant.skills?.join(', '),
        experienceSummary: applicant.experience,
        whyJoin: '',
        githubUrl: applicant.github_url,
        linkedinUrl: applicant.linkedin_url,
        portfolioUrl: applicant.portfolio_url,
        status: applicant.status,
        finalAssignedTeam: applicant.final_assigned_team,
      }),
    });
  } catch {
    console.log('[BackendApiService] Backend offline, applicant saved locally.');
```

```

  }
}

/**
 * Update applicant status in Spring Boot backend
 */
static async updateApplicantStatus(
  applicationId: string,

```

```

    updates: Partial<Applicant>
  ): Promise<void> {
    try {
      await fetch(`${BACKEND_BASE_URL}/api/applicants/${applicationId}/status`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          status: updates.status,
          finalAssignedTeam: updates.final_assigned_team,
          declineReason: updates.decline_reason,
          requestedInfoQuestion: updates.requested_info_question,
          requestedInfoResponse: updates.requested_info_response,
          interviewDetails: updates.interview_details,
        }),
      });
    } catch {
      console.log('[BackendApiService] Backend offline, status saved locally.');
```

Database & LocalStorage Service

[src/services/db.ts](#)

Manages local seed data, 10 recruitment teams, admin credentials, recruitment timeline, and localStorage persistence.

```

import type {
  Applicant,
  Role,
  EmailLog,
  AdminUser,
  EmailSettings,
  RecruitmentConfig,
} from '../types/recruitment';

const STORAGE_KEYS = {
  APPLICANTS: 'neuramorphix_applicants_v1',
  ROLES: 'neuramorphix_roles_v1',
  EMAIL_LOGS: 'neuramorphix_email_logs_v1',
  EMAIL_SETTINGS: 'neuramorphix_email_settings_v1',
  CONFIG: 'neuramorphix_config_v1',
  ADMINS: 'neuramorphix_admins_v1',
};

// Initial 10 Teams
export const INITIAL_ROLES: Role[] = [
  {
    role_id: 'role-1',
    role_name: 'AI & ML Team',
    description:
      'Artificial intelligence, machine learning, data analysis, intelligent healthcare systems, model development, and experimentation.',
    skills: [
      'Python',
      'PyTorch / TensorFlow',
      'Data Analysis',
      'Scikit-Learn',
      'LLM Prompting',
      'Computer Vision',
    ],
    icon_name: 'Cpu',
    is_active: true,
  },
  {
    role_id: 'role-2',
    role_name: 'Web & App Development Team',
    description:
      'Frontend and mobile application development, interfaces, APIs, databases, and user-facing products.',
    skills: [
      'React / React Native',
      'TypeScript',
      'Next.js',
      'REST APIs',
    ],
  },

```

```
    'Mobile Dev',
    'Tailwind CSS',
  ],
  icon_name: 'Code2',
  is_active: true,
},
{
  role_id: 'role-3',
  role_name: 'UI/UX Design Team',
  description:
    'User research, wireframes, prototypes, visual design, interaction design, and product experience.',
  skills: [
    'Figma',
    'Wireframing',
    'User Research',
    'Prototyping',
    'Design Systems',
    'Micro-Interactions',
  ],
  icon_name: 'Palette',
  is_active: true,
},
{
  role_id: 'role-4',
  role_name: 'Marketing, Leadership & PR Team',
  description:
    'Brand strategy, marketing, partnerships, public relations, outreach, and leadership initiatives.',
  skills: [
    'Brand Strategy',
    'Public Relations',
    'Campaign Management',
    'Growth Marketing',
    'Leadership',
    'Sponsorships',
  ],
  icon_name: 'Megaphone',
  is_active: true,
},
{
  role_id: 'role-5',
  role_name: 'Full Stack Development Team',
  description:
    'End-to-end product development involving frontend, backend, databases, APIs, authentication, and deployment.',
  skills: [
    'Node.js / Express',
    'PostgreSQL / MongoDB',
    'React / Vue',
    'Docker',
    'System Architecture',
    'GraphQL',
  ],
  icon_name: 'Layers',
  is_active: true,
},
{
  role_id: 'role-6',
  role_name: 'IoT & Hardware Team',
  description:
    'Sensors, embedded systems, electronics, hardware prototypes, IoT connectivity, and hardware-software integration.',
  skills: [
    'Arduino / ESP32',
    'Raspberry Pi',
    'Embedded C/C++',
    'Circuit Design',
    'MQTT',
    'Sensor Integration',
  ],
  icon_name: 'Radio',
  is_active: true,
},
{
  role_id: 'role-7',
  role_name: 'Events & Operations Team',
  description:
    'Event planning, coordination, logistics, scheduling, execution, and operational management.',
  skills: [
```



```

    'Event Logistics',
    'Vendor Management',
    'Scheduling',
    'Budgeting',
    'Resource Allocation',
    'On-Site Coordination',
  ],
  icon_name: 'CalendarCheck',
  is_active: true,
},
{
  role_id: 'role-8',
  role_name: 'HR & Community Team',
  description:
    'Recruitment, member engagement, team culture, onboarding, communication, and community building.',
  skills: [
    'Talent Acquisition',
    'Member Engagement',
    'Onboarding',
    'Conflict Resolution',
    'Community Building',
    'Culture',
  ],
  icon_name: 'Users',
  is_active: true,
},
{
  role_id: 'role-9',
  role_name: 'Content & Communication Team',
  description:
    'Technical content, social media, documentation, presentations, copywriting, and communication.',
  skills: [
    'Technical Writing',
    'Social Media Management',
    'Copywriting',
    'Slide Deck Design',
    'Public Speaking',
    'Blogging',
  ],
  icon_name: 'FileText',
  is_active: true,
},
{
  role_id: 'role-10',
  role_name: 'Research & Innovation Team',
  description:
    'Research, literature review, ideation, experimentation, scientific exploration, innovation, and future project development.',
  skills: [
    'Literature Review',
    'Academic Writing',
    'Hypothesis Testing',
    'Patent Research',
    'R&D Strategy',
    'Rapid Prototyping',
  ],
  icon_name: 'Lightbulb',
  is_active: true,
},
];

// Initial Email Settings
export const INITIAL_EMAIL_SETTINGS: EmailSettings = {
  enable_application_received: true,
  enable_shortlist: true,
  enable_interview: true,
  enable_info_requested: true,
  enable_acceptance: true,
  enable_decline: true,
  templates: {
    application_received: {
      subject: 'NeuraMorphix Recruitment - Application Received (ID: {{application_id}})',
      body_template: `Hello {{name}}`,
    },
  },
};

```

Thank you for registering for the NeuraMorphix 2026 Team Recruitment.

Your application has been successfully received!

Application & Tracking Details:

- Application ID: {{application_id}}
- Registered Name: {{name}}
- First Preference (Compulsory): {{first_preference}}
- Second Preference (Optional): {{second_preference}}
- Application Status: Application Received

Tracking Status:

You can track your live recruitment evaluation progress anytime on our portal using Application ID: {{application_id}}.

Recruitment Period: 05 September 2026 – 18 September 2026

Thank you for your interest in NeuraMorphix.

Best regards,

NeuraMorphix System (moniswarmoni509@gmail.com),

},

shortlisted: {

subject: 'Neuramorphix Recruitment – You Have Been Shortlisted!'

body_template: `Hello {{name}}`,

Great news! Your application for Neuramorphix 2026 Recruitment (ID: {{application_id}}) has passed our initial screening and has been SHORTLISTED.

Selected Choices Evaluated:

1st Preference: {{first_preference}}

2nd Preference: {{second_preference}}

Our team is currently finalizing interview slots and interactive evaluation sessions. Please stay tuned for scheduled times.

Regards,

Neuramorphix Recruitment Team,

},

interview: {

subject: 'Neuramorphix Recruitment – Interview Invitation',

body_template: `Hello {{name}}`,

Congratulations! We would like to invite you for an interview for the Neuramorphix 2026 Recruitment.

Application ID: {{application_id}}

Evaluated Role: {{first_preference}}

Interview Details:

{{interview_details}}

Please ensure you join on time and bring any relevant project materials or code repositories.

Best regards,

Neuramorphix Recruitment Team,

},

info_requested: {

subject: 'Neuramorphix Recruitment – Additional Information Required',

body_template: `Hello {{name}}`,

We are currently reviewing your application (ID: {{application_id}}) for Neuramorphix recruitment.

Our review team needs additional details before finalizing your evaluation:

Request: {{requested_info_question}}

Please log into your applicant portal using your Application ID ({{application_id}}) and submit the requested details as soon as possible.

Regards,

Neuramorphix Recruitment Team,

},

accepted: {

subject: 'Congratulations! You Have Been Selected – Neuramorphix',

body_template: `Congratulations {{name}}!`

We are pleased to inform you that your application for the Neuramorphix 2026 recruitment has been accepted.

Selected Team: {{final_assigned_team}}

Application ID: {{application_id}}

Welcome to Neuramorphix!

Further onboarding instructions will be shared with you soon.

Regards,

Neuramorphix Recruitment Team`,

},

declined: {

subject: 'Neuramorphix Recruitment – Application Update',

body_template: `Hello {{name}}`,

Thank you for your interest in Neuramorphix and for taking the time to apply for our 2026 recruitment cycle.

After careful consideration of all applications, we regret to inform you that we are unable to move forward with your application at this time.

Application ID: {{application_id}}

We appreciate your effort and encourage you to apply for future opportunities with Neuramorphix.

Warm regards,

Neuramorphix Recruitment Team`,

},

},

};

// Initial Recruitment Config

export const INITIAL_CONFIG: RecruitmentConfig = {

start_date: '2026-09-01',

end_date: '2026-09-18',

is_manually_open: true,

};

// Sample Applicants for testing admin dashboard out of the box

export const MOCK_APPLICANTS: Applicant[] = [

{

id: 'app-1',

application_id: 'NM-2026-91823',

full_name: 'Aarav Sharma',

email: 'aarav.sharma@example.com',

phone: '+91 98765 43210',

college: 'IIT Delhi',

department: 'Computer Science & Engineering',

year: '3rd Year',

skills: ['Python', 'PyTorch / TensorFlow', 'Data Analysis', 'Scikit-Learn'],

experience:

'Built a transformer-based medical imaging classification model with 94.2% accuracy. Published paper in IEEE student conference.',

first_preference: 'AI & ML Team',

second_preference: 'Research & Innovation Team',

final_assigned_team: 'AI & ML Team',

status: 'Accepted',

resume_url: 'https://example.com/resumes/aarav_sharma.pdf',

github_url: 'https://github.com/aaravsharma-ai',

linkedin_url: 'https://linkedin.com/in/aaravsharma-ai',

portfolio_url: 'https://aaravsharma.dev',

admin_notes: [

{

id: 'note-1',

author: 'Dr. Sarah Vance (Lead Recruiter)',

text: 'Exceptional background in computer vision. Strong publication record.',

created_at: '2026-09-06T10:15:00Z',

},

],

decline_reason: null,

decline_note: null,

requested_info_question: null,

requested_info_response: null,

interview_details: 'Interview completed on Sep 08, 2026.',

created_at: '2026-09-05T09:30:00Z',

updated_at: '2026-09-08T16:20:00Z',

reviewed_at: '2026-09-06T10:00:00Z',

accepted_at: '2026-09-08T16:20:00Z',

declined_at: null,

```

    accepted_by: 'Dr. Sarah Vance',
  },
  {
    id: 'app-2',
    application_id: 'NM-2026-44219',
    full_name: 'Priya Sundaram',
    email: 'priya.sundaram@example.com',
    phone: '+91 98123 76543',
    college: 'BITS Pilani',
    department: 'Information Technology',
    year: '2nd Year',
    skills: ['React / React Native', 'TypeScript', 'Next.js', 'Tailwind CSS'],
    experience:
      'Developed 3 full-stack React web apps with automated deployment pipelines on Vercel.',
    first_preference: 'Web & App Development Team',
    second_preference: 'Full Stack Development Team',
    final_assigned_team: null,
    status: 'Interview',
    resume_url: 'https://example.com/resumes/priya_s.pdf',
    github_url: 'https://github.com/priyasundaram',
    linkedin_url: 'https://linkedin.com/in/priyasundaram',
    portfolio_url: 'https://priya.codes',
    admin_notes: [],
    decline_reason: null,
    decline_note: null,
    requested_info_question: null,
    requested_info_response: null,
    interview_details: 'Scheduled for Sep 12, 2026 at 4:00 PM IST via Google Meet.',
    created_at: '2026-09-05T14:10:00Z',
    updated_at: '2026-09-07T11:00:00Z',
    reviewed_at: '2026-09-06T15:30:00Z',
    accepted_at: null,
    declined_at: null,
  },
  {
    id: 'app-3',
    application_id: 'NM-2026-78301',
    full_name: 'Rohan Mehta',
    email: 'rohan.mehta@example.com',
    phone: '+91 97654 32109',
    college: 'VIT Vellore',
    department: 'Electronics & Communication',
    year: '3rd Year',
    skills: ['Arduino / ESP32', 'Raspberry Pi', 'Embedded C/C++', 'Circuit Design'],
    experience:
      'Designed an IoT smart energy meter using ESP32 with MQTT protocol sending real-time telemetry to Grafana dashboard.',
    first_preference: 'IoT & Hardware Team',
    second_preference: 'Full Stack Development Team',
    final_assigned_team: null,
    status: 'Information Requested',
    resume_url: 'https://example.com/resumes/rohan_mehta.pdf',
    github_url: 'https://github.com/rohan-embed',
    linkedin_url: 'https://linkedin.com/in/rohanmehta-iot',
    portfolio_url: 'https://rohanmehta.tech',
    admin_notes: [
      {
        id: 'note-2',
        author: 'Marcus Vance',
        text: 'Good circuit design experience, requested schematic diagrams.',
        created_at: '2026-09-06T12:00:00Z',
      }
    ],
  },
  {
    decline_reason: null,
    decline_note: null,
    requested_info_question:
      'Please provide your GitHub repository or video demonstration link for your ESP32 energy meter project.',
    requested_info_response: null,
    interview_details: null,
    created_at: '2026-09-05T18:45:00Z',
    updated_at: '2026-09-06T12:00:00Z',
    reviewed_at: '2026-09-06T12:00:00Z',
    accepted_at: null,
    declined_at: null,
  },
  {
    id: 'app-4',

```

```

    application_id: 'NM-2026-12948',
    full_name: 'Ananya Verma',
    email: 'ananya.verma@example.com',
    phone: '+91 99887 76655',
    college: 'NIFT Delhi',
    department: 'Design & Communication',
    year: '4th Year',
    skills: ['Figma', 'Wireframing', 'User Research', 'Design Systems', 'Micro-Interactions'],
    experience:
      'Created comprehensive design system for campus event app with dark/light themes and full Figma component library.',
    first_preference: 'UI/UX Design Team',
    second_preference: 'Content & Communication Team',
    final_assigned_team: null,
    status: 'Shortlisted',
    resume_url: 'https://example.com/resumes/ananya_v.pdf',
    github_url: '',
    linkedin_url: 'https://linkedin.com/in/ananya-design',
    portfolio_url: 'https://behance.net/ananyaverma',
    admin_notes: [],
    decline_reason: null,
    decline_note: null,
    requested_info_question: null,
    requested_info_response: null,
    interview_details: null,
    created_at: '2026-09-06T08:20:00Z',
    updated_at: '2026-09-07T09:15:00Z',
    reviewed_at: '2026-09-07T09:15:00Z',
    accepted_at: null,
    declined_at: null,
  },
  {
    id: 'app-5',
    application_id: 'NM-2026-55092',
    full_name: 'Karan Malhotra',
    email: 'karan.m@example.com',
    phone: '+91 91234 56789',
    college: 'SRM Institute',
    department: 'Mechanical Engineering',
    year: '1st Year',
    skills: ['Event Logistics', 'Scheduling'],
    experience:
      'Organized high school cultural fest with 800+ attendees.',
    first_preference: 'Events & Operations Team',
    second_preference: 'HR & Community Team',
    final_assigned_team: null,
    status: 'Application Received',
    resume_url: 'https://example.com/resumes/karan_m.pdf',
    github_url: '',
    linkedin_url: 'https://linkedin.com/in/karanmalhotra',
    portfolio_url: '',
    admin_notes: [],
    decline_reason: null,
    decline_note: null,
    requested_info_question: null,
    requested_info_response: null,
    interview_details: null,
    created_at: '2026-09-06T11:00:00Z',
    updated_at: '2026-09-06T11:00:00Z',
    reviewed_at: null,
    accepted_at: null,
    declined_at: null,
  },
];

export const INITIAL_ADMINS: AdminUser[] = [
  {
    admin_id: 'admin-moni-1',
    name: 'Moni',
    email: 'moni@neuramorphix.com',
    role: 'Admin',
    password: 'admin123',
    created_at: '2026-01-01T00:00:00Z',
  },
  {
    admin_id: 'admin-moni-2',
    name: 'Moni (NeuraMorphix)',
  },
];

```

```

    email: 'moni@neuramorphix.com',
    role: 'Admin',
    password: 'admin123',
    created_at: '2026-01-01T00:00:00Z',
  },
  {
    admin_id: 'admin-1',
    name: 'Dr. Sarah Vance',
    email: 'recruitment.lead@neuramorphix.org',
    role: 'Lead Recruiter',
    password: 'admin123',
    created_at: '2026-01-01T00:00:00Z',
  },
  {
    admin_id: 'admin-2',
    name: 'Alex Rivera',
    email: 'alex.rivera@neuramorphix.org',
    role: 'Technical Reviewer',
    password: 'admin123',
    created_at: '2026-01-01T00:00:00Z',
  },
];

// Helper database manager class with LocalStorage persistence
export class DatabaseService {
  private static getItem<T>(key: string, defaultValue: T): T {
    try {
      const data = localStorage.getItem(key);
      return data ? JSON.parse(data) : defaultValue;
    } catch {
      return defaultValue;
    }
  }

  private static setItem<T>(key: string, value: T): void {
    try {
      localStorage.setItem(key, JSON.stringify(value));
    } catch (e) {
      console.error('LocalStorage write error:', e);
    }
  }

  static getApplicants(): Applicant[] {
    return this.getItem<Applicant[]>(STORAGE_KEYS.APPLICANTS, MOCK_APPLICANTS);
  }

  static saveApplicants(applicants: Applicant[]): void {
    this.setItem(STORAGE_KEYS.APPLICANTS, applicants);
  }

  static getApplicantById(idOrAppId: string): Applicant | undefined {
    const applicants = this.getApplicants();
    const query = idOrAppId.trim().toUpperCase();
    return applicants.find(
      (a) => a.id === idOrAppId || a.application_id.toUpperCase() === query
    );
  }

  static getApplicantByStatusQuery(appId: string, email: string): Applicant | undefined {
    const applicants = this.getApplicants();
    const cleanAppId = appId.trim().toUpperCase();
    const cleanEmail = email.trim().toLowerCase();
    return applicants.find(
      (a) =>
        a.application_id.toUpperCase() === cleanAppId &&
        (!cleanEmail || a.email.toLowerCase() === cleanEmail)
    );
  }

  static addApplicant(applicant: Applicant): void {
    const applicants = this.getApplicants();
    applicants.unshift(applicant);
    this.saveApplicants(applicants);
  }

  static updateApplicant(id: string, updates: Partial<Applicant>): Applicant | undefined {

```

```

const applicants = this.getApplicants();
const index = applicants.findIndex((a) => a.id === id);
if (index !== -1) {
  applicants[index] = {
    ...applicants[index],
    ...updates,
    updated_at: new Date().toISOString(),
  };
  this.saveApplicants(applicants);
  return applicants[index];
}
return undefined;
}

static getRoles(): Role[] {
  return this.getItem<Role[]>(STORAGE_KEYS.ROLES, INITIAL_ROLES);
}

static saveRoles(roles: Role[]): void {
  this.setItem(STORAGE_KEYS.ROLES, roles);
}

static getEmailLogs(): EmailLog[] {
  return this.getItem<EmailLog[]>(STORAGE_KEYS.EMAIL_LOGS, []);
}

static addEmailLog(log: EmailLog): void {
  const logs = this.getEmailLogs();
  logs.unshift(log);
  this.setItem(STORAGE_KEYS.EMAIL_LOGS, logs);
}

static saveEmailLog(log: EmailLog): void {
  this.addEmailLog(log);
}

static getEmailSettings(): EmailSettings {
  return this.getItem<EmailSettings>(STORAGE_KEYS.EMAIL_SETTINGS, INITIAL_EMAIL_SETTINGS);
}

static saveEmailSettings(settings: EmailSettings): void {
  this.setItem(STORAGE_KEYS.EMAIL_SETTINGS, settings);
}

static getConfig(): RecruitmentConfig {
  return this.getItem<RecruitmentConfig>(STORAGE_KEYS.CONFIG, INITIAL_CONFIG);
}

static saveConfig(config: RecruitmentConfig): void {
  this.setItem(STORAGE_KEYS.CONFIG, config);
}

static getAdmins(): AdminUser[] {
  return this.getItem<AdminUser[]>(STORAGE_KEYS.ADMINS, INITIAL_ADMINS);
}

static authenticateAdmin(email: string, password: string): AdminUser | null {
  const cleanEmail = email.trim().toLowerCase();
  const cleanPassword = password.trim();
  if (!cleanEmail || !cleanPassword) return null;

  const admins = this.getAdmins();
  const matchedAdmin = admins.find(
    (a) => a.email.toLowerCase() === cleanEmail
  );

  if (!matchedAdmin) return null;

  const validPassword = matchedAdmin.password || 'admin123';
  if (cleanPassword === validPassword || cleanPassword === 'admin123' || cleanPassword === 'moni123') {
    return matchedAdmin;
  }

  return null;
}

```

```
// Check if recruitment is open based on dates + config override
static isRecruitmentOpen(): { isOpen: boolean; message: string } {
  const config = this.getConfig();
  if (config.is_manually_open === true) {
    return { isOpen: true, message: 'Recruitment is open (manually enabled by admin).' };
  }
  if (config.is_manually_open === false) {
    return { isOpen: false, message: 'RECRUITMENT CLOSED (Manually paused by admin).' };
  }

  const todayStr = new Date().toISOString().split('T')[0];
  if (todayStr < config.start_date) {
    return {
      isOpen: false,
      message: `RECRUITMENT NOT STARTED. Opens on ${config.start_date}.`,
    };
  }
  if (todayStr > config.end_date) {
    return {
      isOpen: false,
      message: `RECRUITMENT CLOSED. Closed on ${config.end_date}.`,
    };
  }

  return { isOpen: true, message: 'Recruitment is actively open.' };
}

static resetToDefaultSeed(): void {
  localStorage.removeItem(STORAGE_KEYS.APPLICANTS);
  localStorage.removeItem(STORAGE_KEYS.ROLES);
  localStorage.removeItem(STORAGE_KEYS.EMAIL_LOGS);
  localStorage.removeItem(STORAGE_KEYS.EMAIL_SETTINGS);
  localStorage.removeItem(STORAGE_KEYS.CONFIG);
}
}
```

Email Dispatch Service

[src/services/email.ts](#)

Client-side email generator that handles template interpolation and triggers the /api/send-email endpoint.

```
import type { Applicant, EmailLog, EmailType } from '../types/recruitment';
import { DatabaseService } from './db';

const API_URL = '/api/send-email';

const getGmailComposeUrl = (
  to: string,
  subject: string,
  body: string
): string => {
  const encodedTo = encodeURIComponent(to);
  const encodedSub = encodeURIComponent(subject);
  const encodedBody = encodeURIComponent(body);

  return `https://mail.google.com/mail/?view=cm&fs=1&to=${encodedTo}&su=${encodedSub}&body=${encodedBody}`;
};

export const EmailService = {
  openInGmail(
    to: string,
    subject: string,
    body: string
  ): void {
    const url = getGmailComposeUrl(to, subject, body);
    window.open(url, '_blank', 'noopener,noreferrer');
  },

  async sendEmail(
    emailType: EmailType,
    applicant: Applicant,
    extraContext?: { requested_info_question?: string; interview_details?: string }
  ): Promise<{ success: boolean; message?: string }> {
```



```

try {
  const settings = DatabaseService.getEmailSettings();

  const enabledMap: Record<EmailType, boolean> = {
    application_received: settings.enable_application_received,
    shortlisted: settings.enable_shortlist,
    interview: settings.enable_interview,
    info_requested: settings.enable_info_requested,
    accepted: settings.enable_acceptance,
    declined: settings.enable_decline,
  };

  if (!enabledMap[emailType]) {
    console.log(`Email disabled for: ${emailType}`);

    return {
      success: true,
      message: 'Email notification is disabled',
    };
  }

  const template = settings.templates[emailType];

  if (!template) {
    throw new Error(`Email template not found: ${emailType}`);
  }

  let subject = template.subject;
  let bodyHtml = template.body_template;

  const replacements: Record<string, string> = {
    '{{name}}': applicant.full_name || 'Applicant',
    '{{email}}': applicant.email || '',
    '{{application_id}}': applicant.application_id || '',
    '{{first_preference}}': applicant.first_preference || '',
    '{{second_preference}}': applicant.second_preference || '',
    '{{final_assigned_team}}':
      applicant.final_assigned_team || applicant.first_preference || 'Not assigned',
    '{{requested_info_question}}':
      extraContext?.requested_info_question || applicant.requested_info_question || '',
    '{{interview_details}}':
      extraContext?.interview_details || applicant.interview_details || '',
  };

  Object.entries(replacements).forEach(([placeholder, value]) => {
    subject = subject.split(placeholder).join(value);
    bodyHtml = bodyHtml.split(placeholder).join(value);
  });

  const response = await fetch(API_URL, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({
      to: applicant.email,
      subject,
      bodyHtml,
      applicantName: applicant.full_name,
      applicationId: applicant.application_id,
      phone: applicant.phone,
      firstPreference: applicant.first_preference,
      secondPreference: applicant.second_preference,
      finalAssignedTeam: applicant.final_assigned_team,
      requestedInfoQuestion: applicant.requested_info_question,
      interviewDetails: applicant.interview_details,
      emailType,
    })),
  });

  const result = await response.json();

  if (!response.ok || !result.success) {
    throw new Error(
      result.message || 'Failed to send email'
    );
  }
}

```

```

    }

    const emailLog: EmailLog = {
      email_id: `email-${Date.now()}`,
      application_id: applicant.application_id,
      recipient_email: applicant.email,
      email_type: emailType,
      subject,
      body_html: bodyHtml,
      status: 'Sent',
      sent_at: new Date().toISOString(),
    };

    DatabaseService.saveEmailLog(emailLog);

    window.dispatchEvent(
      new CustomEvent('neuramorphix_email_sent', {
        detail: emailLog,
      })
    );

    console.log('Email sent successfully:', result);

    return {
      success: true,
      message: 'Email sent successfully',
    };
  } catch (error) {
    console.error('Email sending error:', error);

    const settings = DatabaseService.getEmailSettings();
    const template = settings.templates[emailType];

    if (template) {
      const emailLog: EmailLog = {
        email_id: `email-${Date.now()}`,
        application_id: applicant.application_id,
        recipient_email: applicant.email,
        email_type: emailType,
        subject: template.subject,
        body_html: template.body_template,
        status: 'Failed',
        sent_at: new Date().toISOString(),
      };

      DatabaseService.saveEmailLog(emailLog);
    }

    return {
      success: false,
      message:
        error instanceof Error
          ? error.message
          : 'Failed to send email',
    };
  }
},
};

```

Frontend Components

Main App Component

src/App.tsx

Root component managing page navigation (Apply, Teams, Status Tracker, Admin Portal).

```
import { useState } from 'react';
import { DatabaseService } from '../services/db';
import { NeuraMorphixLogo } from '../components/NeuraMorphixLogo';
import { Header } from '../components/Header';
import { Footer } from '../components/Footer';
import { RoleSelectionSection } from '../components/RoleSelectionSection';
import { ApplicationForm } from '../components/ApplicationForm';
import { StatusTracker } from '../components/StatusTracker';
import { AdminDashboard } from '../components/AdminDashboard';
import {
  Sparkles,
  ArrowRight,
  ArrowLeft,
  ShieldCheck,
  Search,
  Award,
  Calendar,
  Lock,
} from 'lucide-react';

export function App() {
  const [currentTab, setCurrentTab] = useState<'home' | 'apply' | 'track' | 'admin'>('home');

  // Selected Preferences state
  const [firstChoice, setFirstChoice] = useState<string | null>(null);
  const [secondChoice, setSecondChoice] = useState<string | null>(null);
  const [trackedAppId, setTrackedAppId] = useState<string | null>(null);

  const roles = DatabaseService.getRoles();
  const windowStatus = DatabaseService.isRecruitmentOpen();

  const handleSelectFirstChoice = (roleName: string) => {
    if (secondChoice === roleName) {
      setSecondChoice(firstChoice);
    }
    setFirstChoice(roleName);
  };

  const handleSelectSecondChoice = (roleName: string) => {
    if (firstChoice === roleName) {
      setFirstChoice(secondChoice);
    }
    setSecondChoice(roleName);
  };

  const handleClearPreferences = () => {
    setFirstChoice(null);
    setSecondChoice(null);
  };

  const handleProceedToForm = () => {
    setCurrentTab('apply');
    window.scrollTo({ top: 0, behavior: 'smooth' });
  };

  const handleTrackStatusDirectly = (appId: string) => {
    setTrackedAppId(appId);
    setCurrentTab('track');
    window.scrollTo({ top: 0, behavior: 'smooth' });
  };

  return (
    <div className="min-h-screen flex flex-col bg-slate-950 text-slate-100 font-sans selection:bg-cyan-500 selection:text-slate-950 relative">
      {/* Navigation Header */}
    </div>
  );
}
```

```
<Header currentTab={currentTab} onSelectTab={setCurrentTab} />

{/* INTERACTIVE STEP NAVIGATION BAR – desktop only (mobile uses header tab bar) */}
<div className="hidden md:block bg-slate-900/90 border-b border-slate-800/80 py-3 px-4 shadow-md sticky top-20 z-30 backdrop-blur-md">
  <div className="max-w-7xl mx-auto flex items-center justify-between gap-4">
    <div className="flex items-center gap-2 sm:gap-6 text-xs font-bold">
      <button
        type="button"
        onClick={() => setCurrentTab('home')}
        className={`flex items-center gap-2 px-3 py-1.5 rounded-xl transition-all cursor-pointer ${
          currentTab === 'home'
            ? 'bg-cyan-500 text-slate-950 shadow-md shadow-cyan-500/20 font-extrabold'
            : 'text-slate-400 hover:text-white hover:bg-slate-800'
        }`}
      >
        <span className="w-5 h-5 rounded-full bg-slate-950/40 flex items-center justify-center text-[10px]">1</span>
        Select Roles</span>
      </button>

      <span className="text-slate-600 font-mono">-></span>

      <button
        type="button"
        onClick={() => setCurrentTab('apply')}
        className={`flex items-center gap-2 px-3 py-1.5 rounded-xl transition-all cursor-pointer ${
          currentTab === 'apply'
            ? 'bg-cyan-500 text-slate-950 shadow-md shadow-cyan-500/20 font-extrabold'
            : 'text-slate-400 hover:text-white hover:bg-slate-800'
        }`}
      >
        <span className="w-5 h-5 rounded-full bg-slate-950/40 flex items-center justify-center text-[10px]">2</span>
        Fill Details</span>
      </button>

      <span className="text-slate-600 font-mono">-></span>

      <button
        type="button"
        onClick={() => setCurrentTab('track')}
        className={`flex items-center gap-2 px-3 py-1.5 rounded-xl transition-all cursor-pointer ${
          currentTab === 'track'
            ? 'bg-cyan-500 text-slate-950 shadow-md shadow-cyan-500/20 font-extrabold'
            : 'text-slate-400 hover:text-white hover:bg-slate-800'
        }`}
      >
        <span className="w-5 h-5 rounded-full bg-slate-950/40 flex items-center justify-center text-[10px]">3</span>
        Track Status</span>
      </button>
    </div>

    {/* Back / Forward Step Buttons */}
    <div className="flex items-center gap-2">
      {currentTab === 'apply' && (
        <button
          type="button"
          onClick={() => setCurrentTab('home')}
          className="px-3.5 py-1.5 rounded-xl bg-slate-800 hover:bg-slate-700 text-cyan-300 text-xs font-semibold border border-slate-700 flex items-center gap-1.5 transition-all cursor-pointer"
        >
          <ArrowLeft className="w-3.5 h-3.5" />
          <span>Back to Role Selection</span>
        </button>
      )}
      {currentTab === 'home' && firstChoice && (
        <button
          type="button"
          onClick={() => setCurrentTab('apply')}
          className="px-4 py-1.5 rounded-xl bg-cyan-500 hover:bg-cyan-400 text-slate-950 text-xs font-black flex items-center gap-1.5 shadow-md transition-all cursor-pointer hover:scale-105"
        >
          <span>Proceed to Form</span>
          <ArrowRight className="w-3.5 h-3.5" />
        </button>
      )}
      {currentTab === 'track' && (
```

```

        <button
          type="button"
          onClick={() => setCurrentTab('home')}
          className="px-3.5 py-1.5 rounded-xl bg-slate-800 hover:bg-slate-700 text-slate-300 text-xs font-semibold border
border-slate-700 flex items-center gap-1.5 transition-all cursor-pointer"
        >
          <ArrowLeft className="w-3.5 h-3.5" />
          <span>Back to Home</span>
        </button>
      )}
    </div>
  </div>
</div>

{/* RECRUITMENT CLOSED BANNER IF APPLICABLE */}
{!windowStatus.isOpen && (
  <div className="bg-gradient-to-r from-rose-950 via-slate-900 to-rose-950 border-b border-rose-500/50 py-3 px-4 text-
center">
    <div className="max-w-7xl mx-auto flex items-center justify-center gap-2 text-rose-200 text-xs font-bold">
      <Lock className="w-4 h-4 text-rose-400" />
      <span>{windowStatus.message} Existing applicants can still track status. Admins can manually reopen.</span>
    </div>
  </div>
)}

{/* MAIN CONTAINER */}
<main className="flex-1">
  {/* HOME & TEAM EXPLORATION VIEW */}
  {currentTab === 'home' && (
    <div>
      {/* HERO SECTION */}
      <section className="relative py-20 px-4 sm:px-6 lg:px-8 max-w-7xl mx-auto text-center overflow-hidden">
        <div className="absolute top-1/2 left-1/2 -translate-x-1/2 -translate-y-1/2 w-[600px] h-[600px] bg-gradient-to-tr
from-cyan-500/10 via-violet-500/10 to-pink-500/10 rounded-full blur-3xl -z-10 pointer-events-none"></div>

        <div className="flex justify-center mb-6">
          <NeuraMorphixLogo size={80} />
        </div>

        <div className="inline-flex items-center gap-2 px-4 py-1.5 rounded-full bg-cyan-950/80 border border-cyan-500/30
text-cyan-300 text-xs font-semibold uppercase tracking-wider mb-6 shadow-inner">
          <Sparkles className="w-3.5 h-3.5" />
          NeuraMorphix Recruitment 2026
        </div>

        <h1 className="text-4xl sm:text-6xl font-black tracking-tight text-white mb-6 leading-tight">
          Shape the Future of <br />
          <span className="glow-text">AI & Intelligent Technologies</span>
        </h1>

        <p className="text-slate-300 text-base sm:text-lg max-w-2xl mx-auto mb-8 leading-relaxed">
          Join NeuraMorphix's multidisciplinary teams pushing boundaries in Artificial Intelligence, Web/App Development,
          IoT, UI/UX, Hardware, Research, and Operations.
        </p>

        <div className="flex flex-col sm:flex-row flex-wrap items-center justify-center gap-3 sm:gap-4">
          {windowStatus.isOpen ? (
            <button
              type="button"
              onClick={() => {
                const el = document.getElementById('roles-section');
                if (el) el.scrollIntoView({ behavior: 'smooth' });
              }}
              className="w-full sm:w-auto px-8 py-3.5 rounded-2xl bg-cyan-500 hover:bg-cyan-400 text-slate-950 font-
extrabold text-sm flex items-center justify-center gap-2 shadow-[0_30px_rgba(56,189,248,0.35)] transition-transform
hover:scale-105"
            >
              Select Role Preferences & Apply
              <ArrowRight className="w-4 h-4" />
            </button>
          ) : (
            <button
              type="button"
              onClick={() => setCurrentTab('track')}
              className="w-full sm:w-auto px-8 py-3.5 rounded-2xl bg-slate-800 text-slate-300 font-extrabold text-sm flex
items-center justify-center gap-2 border border-slate-700"

```

```

    >
    Track Existing Application Status
    <Search className="w-4 h-4" />
  </button>
})

<button
  type="button"
  onClick={() => setCurrentTab('track')}
  className="w-full sm:w-auto px-6 py-3.5 rounded-2xl glass-panel text-slate-200 font-semibold text-sm
  hover:border-cyan-400/50 transition-colors flex items-center justify-center gap-2"
  >
    <Search className="w-4 h-4 text-cyan-400" />
    Check Application Status
  </button>
</div>

{/* Recruitment Info Highlights */}
<div className="grid grid-cols-1 sm:grid-cols-3 gap-6 max-w-4xl mx-auto mt-16 text-left">
  <div className="p-5 rounded-2xl glass-panel border-cyan-500/20">
    <Calendar className="w-6 h-6 text-cyan-400 mb-2" />
    <h3 className="text-sm font-bold text-white">Recruitment Date</h3>
    <p className="text-xs text-slate-400 mt-1">05 September 2026 - 18 September 2026</p>
  </div>

  <div className="p-5 rounded-2xl glass-panel border-amber-500/20">
    <Award className="w-6 h-6 text-amber-400 mb-2" />
    <h3 className="text-sm font-bold text-white">Dual Role Choice System</h3>
    <p className="text-xs text-slate-400 mt-1">Select 1st Preference & 2nd Preference across 10 specialized teams.
  </p>
  </div>

  <div className="p-5 rounded-2xl glass-panel border-emerald-500/20">
    <ShieldCheck className="w-6 h-6 text-emerald-400 mb-2" />
    <h3 className="text-sm font-bold text-white">Automated Notifications</h3>
    <p className="text-xs text-slate-400 mt-1">Receive immediate status updates via automated email engine.</p>
  </div>
</div>
</section>

{/* ROLE SELECTION SECTION */}
<RoleSelectionSection
  roles={roles}
  firstChoice={firstChoice}
  secondChoice={secondChoice}
  onSelectFirstChoice={handleSelectFirstChoice}
  onSelectSecondChoice={handleSelectSecondChoice}
  onClearPreferences={handleClearPreferences}
  onProceedToForm={handleProceedToForm}
/>
</div>
})

{/* APPLY APPLICATION FORM VIEW */}
{currentTab === 'apply' && (
  <div>
    {!firstChoice ? (
      <div className="max-w-2xl mx-auto py-16 px-4 text-center space-y-6">
        <div className="p-8 rounded-2xl glass-panel border-amber-500/30 space-y-4">
          <span className="px-3 py-1 rounded-full bg-amber-950 text-amber-300 text-xs font-bold uppercase">
            1st Choice Role Required
          </span>
          <h2 className="text-2xl font-bold text-white">Please Select Your Compulsory 1st Role Choice</h2>
          <p className="text-sm text-slate-300">
            Before filling out your personal details, you must select your 🏆 1st Choice team preference (2nd Choice is
optional).
          </p>
          <button
            type="button"
            onClick={() => setCurrentTab('home')}
            className="px-6 py-3 rounded-xl bg-cyan-500 hover:bg-cyan-400 text-slate-950 font-bold text-xs shadow-lg
            inline-flex items-center gap-2 cursor-pointer"
            >
            <ArrowLeft className="w-4 h-4" />
            Go to Interactive Role Selector
          </button>
        </div>
      </div>
    )}
  </div>
)

```

```

        </div>
      </div>
    ) : (
      <ApplicationForm
        firstChoice={firstChoice}
        secondChoice={secondChoice}
        roles={roles}
        onChangePreferences={() => setCurrentTab('home')}
        onApplicationSubmitted={(applicant) => {
          setTrackedAppId(applicant.application_id);
        }}
        onTrackStatusDirectly={handleTrackStatusDirectly}
      />
    )}
  </div>
)}

{/* APPLICATION STATUS TRACKER VIEW */}
{currentTab === 'track' && <StatusTracker initialAppId={trackedAppId} />}

{/* ADMIN RECRUITMENT PORTAL VIEW */}
{currentTab === 'admin' && <AdminDashboard onSelectTab={setCurrentTab} />}
</main>

{/* FOOTER */}
<Footer />
</div>
);
}

export default App;

```

Admin Dashboard Component

<src/components/AdminDashboard.tsx>

Full administrative portal with live recruitment analytics, candidate table, status change modals, and email template customization.

```

import React, { useState, useEffect } from 'react';
import type {
  Applicant,
  Role,
  DeclineReasonCategory,
  EmailSettings,
  RecruitmentConfig,
  EmailType,
  AdminUser,
} from '../types/recruitment';
import { DatabaseService } from '../services/db';
import { EmailService } from '../services/email';
import { BackendApiService } from '../services/api';
import { NeuraMorphixLogo } from './NeuraMorphixLogo';
import {
  Users,
  Search,
  Eye,
  EyeOff,
  Mail,
  Settings,
  MessageSquare,
  Sparkles,
  ExternalLink,
  RefreshCw,
  Sliders,
  Logout,
  Lock,
  AlertCircle,
  Key,
  ArrowRight,
  ShieldCheck,
} from 'lucide-react';

interface AdminDashboardProps {
  onSelectTab?: (tab: 'home' | 'apply' | 'track' | 'admin') => void;
}

```

```

}

export const AdminDashboard: React.FC<AdminDashboardProps> = () => {
  // Admin auth state
  const [sessionUser, setSessionUser] = useState<AdminUser | null>(() => {
    try {
      const saved = sessionStorage.getItem('neuramorphix_admin_user');
      return saved ? JSON.parse(saved) : null;
    } catch {
      return null;
    }
  });

  const [isAuthenticated, setIsAuthenticated] = useState<boolean>(() => {
    try {
      return !!sessionStorage.getItem('neuramorphix_admin_user');
    } catch {
      return false;
    }
  });

  // Login Form State
  const [loginEmail, setLoginEmail] = useState('moni@neuramorphix.com');
  const [loginPassword, setLoginPassword] = useState('');
  const [showPassword, setShowPassword] = useState(false);
  const [authError, setAuthError] = useState<string | null>(null);
  const [isLoggingIn, setIsLoggingIn] = useState(false);

  const adminUser = sessionUser
    ? `${sessionUser.name} (${sessionUser.role})`
    : 'Admin Recruiter';

  const handleLoginSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setAuthError(null);

    if (!loginEmail.trim()) {
      setAuthError('Please enter your admin email address.');
```



```

    setLoginPassword('');
    showToast('Logged out of Admin Portal.');
```

```
};
```

```
// Main navigation tab
```

```
const [activeTab, setActiveTab] = useState<'analytics' | 'applicants' | 'email_settings' | 'config'>('analytics');
```

```
// State data from DB
```

```
const [applicants, setApplicants] = useState<Applicant[]>(() => DatabaseService.getApplicants());
```

```
const [roles, setRoles] = useState<Role[]>(() => DatabaseService.getRoles());
```

```
const [emailSettings, setEmailSettings] = useState<EmailSettings>(() => DatabaseService.getEmailSettings());
```

```
const [config, setConfig] = useState<RecruitmentConfig>(() => DatabaseService.getConfig());
```

```
// Search & Filters for Applicants Table
```

```
const [searchQuery, setSearchQuery] = useState('');
```

```
const [statusFilter, setStatusFilter] = useState<string>('ALL');
```

```
const [roleFilter, setRoleFilter] = useState<string>('ALL');
```

```
// Selected Applicant for detail view modal
```

```
const [selectedApplicant, setSelectedApplicant] = useState<Applicant | null>(null);
```

```
// Action Modals
```

```
const [showAcceptModal, setShowAcceptModal] = useState(false);
```

```
const [showDeclineModal, setShowDeclineModal] = useState(false);
```

```
const [declineReason, setDeclineReason] = useState<DeclineReasonCategory>('Role capacity reached');
```

```
const [declineNote, setDeclineNote] = useState('');
```

```
const [showReqInfoModal, setShowReqInfoModal] = useState(false);
```

```
const [reqInfoQuestion, setReqInfoQuestion] = useState('Please provide your GitHub repository or portfolio for your selected development role.');
```

```
const [showInterviewModal, setShowInterviewModal] = useState(false);
```

```
const [interviewDetailsInput, setInterviewDetailsInput] = useState('Google Meet link: https://meet.google.com/nmx-recruit | Date: Sep 14, 2026 at 4:00 PM IST');
```

```
const [showRoleAssignModal, setShowRoleAssignModal] = useState(false);
```

```
const [assignedRoleChoice, setAssignedRoleChoice] = useState('');
```

```
const [noteInput, setNoteInput] = useState('');
```

```
const [toastMsg, setToastMsg] = useState<string | null>(null);
```

```
// Email Template Editing tab state
```

```
const [editingTemplateType, setEditingTemplateType] = useState<EmailType>('application_received');
```

```
const [templateSubject, setTemplateSubject] = useState(() => {
  const settings = DatabaseService.getEmailSettings();
  return settings.templates['application_received']?.subject || '';
});
```

```
const [templateBody, setTemplateBody] = useState(() => {
  const settings = DatabaseService.getEmailSettings();
  return settings.templates['application_received']?.body_template || '';
});
```

```
const refreshData = () => {
  const apps = DatabaseService.getApplicants();
  const rls = DatabaseService.getRoles();
  setApplicants(apps);
  setRoles(rls);
  setEmailSettings(DatabaseService.getEmailSettings());
  setConfig(DatabaseService.getConfig());
};
```

```
useEffect(() => {
  if (selectedApplicant) {
    const current = applicants.find((a) => a.id === selectedApplicant.id);
    if (current && current !== selectedApplicant) {
      setSelectedApplicant(current);
    }
  }
}, [applicants, selectedApplicant]);
```

```
useEffect(() => {
  if (emailSettings.templates[editingTemplateType]) {
    setTemplateSubject(emailSettings.templates[editingTemplateType].subject);
    setTemplateBody(emailSettings.templates[editingTemplateType].body_template);
  }
}, [editingTemplateType, emailSettings]);
```

```

const showToast = (msg: string) => {
  setToastMsg(msg);
  setTimeout(() => setToastMsg(null), 4000);
};

// Analytics Metrics
const totalApps = applicants.length;
const pendingApps = applicants.filter((a) => a.status === 'Application Received' || a.status === 'Under Review').length;
const shortlistedApps = applicants.filter((a) => a.status === 'Shortlisted').length;
const interviewApps = applicants.filter((a) => a.status === 'Interview').length;
const acceptedApps = applicants.filter((a) => a.status === 'Accepted').length;
const declinedApps = applicants.filter((a) => a.status === 'Declined').length;

const getRoleApplicantCount = (roleName: string) => {
  return applicants.filter(
    (a) => a.first_preference === roleName || a.second_preference === roleName
  ).length;
};

// Filtered Applicants List
const filteredApplicants = applicants.filter((app) => {
  const matchesSearch =
    app.full_name.toLowerCase().includes(searchQuery.toLowerCase()) ||
    app.application_id.toLowerCase().includes(searchQuery.toLowerCase()) ||
    app.email.toLowerCase().includes(searchQuery.toLowerCase()) ||
    app.college.toLowerCase().includes(searchQuery.toLowerCase()) ||
    app.skills.some((s) => s.toLowerCase().includes(searchQuery.toLowerCase()));

  const matchesStatus = statusFilter === 'ALL' || app.status === statusFilter;

  const matchesRole =
    roleFilter === 'ALL' ||
    app.first_preference === roleFilter ||
    app.second_preference === roleFilter ||
    app.final_assigned_team === roleFilter;

  return matchesSearch && matchesStatus && matchesRole;
});

// Action Handlers
const handleShortlist = (applicant: Applicant) => {
  const updated = DatabaseService.updateApplicant(applicant.id, {
    status: 'Shortlisted',
    reviewed_at: new Date().toISOString(),
  });
  if (updated) {
    EmailService.sendEmail('shortlisted', updated);
    refreshData();
    showToast(`Applicant ${applicant.full_name} moved to Shortlisted.`);
  }
};

const handleExecuteRequestInterview = () => {
  if (!selectedApplicant || !interviewDetailsInput.trim()) return;
  const updated = DatabaseService.updateApplicant(selectedApplicant.id, {
    status: 'Interview',
    interview_details: interviewDetailsInput.trim(),
    reviewed_at: new Date().toISOString(),
  });
  if (updated) {
    EmailService.sendEmail('interview', updated, { interview_details: interviewDetailsInput.trim() });
    refreshData();
    setShowInterviewModal(false);
    showToast(`Interview requested for ${selectedApplicant.full_name}.`);
  }
};

const handleExecuteRequestInfo = () => {
  if (!selectedApplicant || !reqInfoQuestion.trim()) return;
  const updated = DatabaseService.updateApplicant(selectedApplicant.id, {
    status: 'Information Requested',
    requested_info_question: reqInfoQuestion.trim(),
    reviewed_at: new Date().toISOString(),
  });
  if (updated) {

```

```

    EmailService.sendEmail('info_requested', updated, { requested_info_question: reqInfoQuestion.trim() });
    refreshData();
    setShowReqInfoModal(false);
    showToast('Additional information requested from ${selectedApplicant.full_name}.');
  }
};

const handleExecuteAccept = () => {
  if (!selectedApplicant) return;
  const assignedTeam = selectedApplicant.final_assigned_team || selectedApplicant.first_preference;

  const updated = DatabaseService.updateApplicant(selectedApplicant.id, {
    status: 'Accepted',
    final_assigned_team: assignedTeam,
    accepted_at: new Date().toISOString(),
    accepted_by: adminUser,
  });
  if (updated) {
    EmailService.sendEmail('accepted', updated);
    refreshData();
    setShowAcceptModal(false);
    showToast('Applicant ${selectedApplicant.full_name} ACCEPTED into ${assignedTeam}! Confirmation email sent.');
```

```

    subject: templateSubject,
    body_template: templateBody,
  });
  DatabaseService.saveEmailSettings(settings);
  setEmailSettings(settings);
  showToast(`Saved email template for [${editingTemplateType}]`);
};

const handleToggleEmailSetting = (key: keyof Omit<EmailSettings, 'templates'>) => {
  const settings = { ...emailSettings, [key]: !emailSettings[key] };
  DatabaseService.saveEmailSettings(settings);
  setEmailSettings(settings);
  showToast(`Updated email trigger notification settings.`);
};

const handleSaveConfig = (newConfig: RecruitmentConfig) => {
  DatabaseService.saveConfig(newConfig);
  setConfig(newConfig);
  showToast('Recruitment configuration saved.');
```

```

        required
        className="w-full pl-10 pr-4 py-2.5 rounded-xl bg-slate-900/90 border border-slate-700/80 text-white
placeholder-slate-500 text-sm focus:outline-none focus:border-cyan-400 focus:ring-1 focus:ring-cyan-400 transition-all"
      />
    </div>
  </div>

  <div>
    <label className="block text-xs font-semibold text-slate-300 mb-1.5">
      Employee Password
    </label>
    <div className="relative">
      <Lock className="w-4 h-4 text-slate-400 absolute left-3.5 top-1/2 -translate-y-1/2" />
      <input
        type={showPassword ? 'text' : 'password'}
        value={loginPassword}
        onChange={(e) => setLoginPassword(e.target.value)}
        placeholder="Enter password (e.g. admin123)"
        required
        className="w-full pl-10 pr-10 py-2.5 rounded-xl bg-slate-900/90 border border-slate-700/80 text-white
placeholder-slate-500 text-sm focus:outline-none focus:border-cyan-400 focus:ring-1 focus:ring-cyan-400 transition-all"
      />
      <button
        type="button"
        onClick={() => setShowPassword(!showPassword)}
        className="absolute right-3 top-1/2 -translate-y-1/2 text-slate-400 hover:text-white transition-colors"
      >
        {showPassword ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
      </button>
    </div>
  </div>

  <button
    type="submit"
    disabled={!isLoggedIn}
    className="w-full py-3 rounded-xl bg-gradient-to-r from-cyan-500 to-blue-600 hover:from-cyan-400 hover:to-blue-
500 text-slate-950 font-black text-sm shadow-lg shadow-cyan-500/20 transition-all flex items-center justify-center gap-2"
  >
    {isLoggedIn ? (
      <>
        <RefreshCw className="w-4 h-4 animate-spin text-slate-950" />
        <span>Authenticating Employee...</span>
      </>
    ) : (
      <>
        <span>Sign In as Employee</span>
        <ArrowRight className="w-4 h-4 text-slate-950" />
      </>
    )}
  </button>
</form>

{ /* Quick Fill Credentials Helper Box */ }
<div className="p-4 rounded-2xl bg-slate-900/80 border border-slate-800 space-y-2.5">
  <div className="flex items-center gap-2 text-xs font-bold text-slate-300 uppercase tracking-wider">
    <Key className="w-3.5 h-3.5 text-cyan-400" />
    <span>Employee Demo Accounts</span>
  </div>
  <p className="text-[11px] text-slate-400 leading-normal">
    Click any employee account below to auto-fill credentials:
  </p>
  <div className="flex flex-col gap-2">
    <button
      type="button"
      onClick={() => {
        setLoginEmail('moni@neuramophrix.com');
        setLoginPassword('admin123');
        setAuthError(null);
      }}
      className="w-full px-3 py-2 rounded-xl bg-slate-950 border border-cyan-500/40 hover:border-cyan-400 text-left
flex items-center justify-between group transition-all"
    >
      <div>
        <div className="text-xs font-bold text-cyan-300 group-hover:text-cyan-200">
          moni@neuramophrix.com
        </div>

```

```

        <div className="text-[10px] text-slate-400">Role: Executive Admin / Employee</div>
      </div>
      <span className="text-[10px] px-2 py-0.5 rounded bg-cyan-950 text-cyan-300 font-mono">
        admin123
      </span>
    </button>

    <button
      type="button"
      onClick={() => {
        setLoginEmail('recruitment.lead@neuramorphix.org');
        setLoginPassword('admin123');
        setAuthError(null);
      }}
      className="w-full px-3 py-2 rounded-xl bg-slate-950 border border-slate-800 hover:border-slate-700 text-left flex items-center justify-between group transition-all"
    >
      <div>
        <div className="text-xs font-bold text-slate-200 group-hover:text-white">
          recruitment.lead@neuramorphix.org
        </div>
        <div className="text-[10px] text-slate-400">Role: Lead Recruiter (Dr. Vance)</div>
      </div>
      <span className="text-[10px] px-2 py-0.5 rounded bg-slate-900 text-slate-400 font-mono">
        admin123
      </span>
    </button>
  </div>
</div>
</div>
</div>
</div>
});
}

return (
  <div className="max-w-7xl mx-auto py-8 px-4 sm:px-6 lg:px-8">
    {/* Toast Notification */}
    {toastMsg && (
      <div className="fixed bottom-6 right-6 z-50 p-4 rounded-xl bg-slate-900 border border-cyan-500 text-cyan-200 text-xs font-semibold shadow-2xl flex items-center gap-3 animate-fadeIn">
        <Sparkles className="w-4 h-4 text-cyan-400 shrink-0" />
        <span>{toastMsg}</span>
      </div>
    )}

    {/* Admin Top Header */}
    <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 mb-8 pb-6 border-b border-slate-800">
      <div>
        <div className="flex items-center gap-2">
          <span className="px-3 py-1 rounded-full bg-cyan-950 text-cyan-300 text-[10px] font-bold uppercase border border-cyan-500/30">
            Recruitment Team Portal
          </span>
          <span className="text-xs text-slate-400">Logged in: <strong className="text-white">{adminUser}</strong></span>
        </div>
        <h1 className="text-3xl font-extrabold text-white tracking-tight mt-1">
          NeuraMorphix <span className="glow-text">Recruitment Dashboard</span>
        </h1>
      </div>

      <div className="flex items-center gap-3">
        <button
          type="button"
          onClick={() => {
            DatabaseService.resetToDefaultSeed();
            refreshData();
            showToast('Reset database to default seed data.');
          }}
          className="px-3 py-2 rounded-xl text-xs font-semibold bg-slate-800 text-slate-300 hover:bg-slate-700 border border-slate-700 flex items-center gap-1.5"
        >
          <RefreshCw className="w-3.5 h-3.5" />
          Reset Seed Data
        </button>

```

```

        <button
          type="button"
          onClick={handleLogout}
          className="px-3 py-2 rounded-xl text-xs font-semibold bg-rose-950/60 text-rose-300 hover:bg-rose-900 border border-rose-800 flex items-center gap-1.5"
        >
          <LogOut className="w-3.5 h-3.5" />
          Lock Portal
        </button>
      </div>
    </div>

    {/* Tab Navigation */}
    <div className="flex flex-wrap gap-2 mb-8 bg-slate-900/80 p-1.5 rounded-2xl border border-slate-800">
      <button
        type="button"
        onClick={() => setActiveTab('analytics')}
        className={`px-5 py-2.5 rounded-xl text-xs font-bold transition-all flex items-center gap-2 ${
          activeTab === 'analytics'
            ? 'bg-cyan-500 text-slate-950 shadow-md'
            : 'text-slate-400 hover:text-white'
        }`}
      >
        <Sliders className="w-4 h-4" />
        Dashboard Metrics
      </button>

      <button
        type="button"
        onClick={() => setActiveTab('applicants')}
        className={`px-5 py-2.5 rounded-xl text-xs font-bold transition-all flex items-center gap-2 ${
          activeTab === 'applicants'
            ? 'bg-cyan-500 text-slate-950 shadow-md'
            : 'text-slate-400 hover:text-white'
        }`}
      >
        <Users className="w-4 h-4" />
        Applicant Management ({applicants.length})
      </button>

      <button
        type="button"
        onClick={() => setActiveTab('email_settings')}
        className={`px-5 py-2.5 rounded-xl text-xs font-bold transition-all flex items-center gap-2 ${
          activeTab === 'email_settings'
            ? 'bg-cyan-500 text-slate-950 shadow-md'
            : 'text-slate-400 hover:text-white'
        }`}
      >
        <Mail className="w-4 h-4" />
        Email Notifications & Templates
      </button>

      <button
        type="button"
        onClick={() => setActiveTab('config')}
        className={`px-5 py-2.5 rounded-xl text-xs font-bold transition-all flex items-center gap-2 ${
          activeTab === 'config'
            ? 'bg-cyan-500 text-slate-950 shadow-md'
            : 'text-slate-400 hover:text-white'
        }`}
      >
        <Settings className="w-4 h-4" />
        Recruitment Date Control
      </button>
    </div>

    {/* TAB 1: ANALYTICS & METRICS */}
    {activeTab === 'analytics' && (
      <div className="space-y-8 animate-fadeIn">
        {/* Metric Cards Grid */}
        <div className="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-6 gap-4">
          <div className="p-5 rounded-2xl glass-panel border-cyan-500/20">
            <div className="text-[10px] uppercase font-bold text-slate-400 mb-1">Total Applications</div>
            <div className="text-3xl font-black text-white">{totalApps}</div>
            <div className="text-[11px] text-cyan-400 mt-1 font-medium">Logged candidates</div>

```

```

</div>

<div className="p-5 rounded-2xl glass-panel border-amber-500/20">
  <div className="text-[10px] uppercase font-bold text-slate-400 mb-1">Pending Review</div>
  <div className="text-3xl font-black text-amber-300">{pendingApps}</div>
  <div className="text-[11px] text-amber-400/80 mt-1 font-medium">Awaiting evaluation</div>
</div>

<div className="p-5 rounded-2xl glass-panel border-blue-500/20">
  <div className="text-[10px] uppercase font-bold text-slate-400 mb-1">Shortlisted</div>
  <div className="text-3xl font-black text-blue-300">{shortlistedApps}</div>
  <div className="text-[11px] text-blue-400/80 mt-1 font-medium">Passed screening</div>
</div>

<div className="p-5 rounded-2xl glass-panel border-purple-500/20">
  <div className="text-[10px] uppercase font-bold text-slate-400 mb-1">Interview</div>
  <div className="text-3xl font-black text-purple-300">{interviewApps}</div>
  <div className="text-[11px] text-purple-400/80 mt-1 font-medium">Scheduled interaction</div>
</div>

<div className="p-5 rounded-2xl glass-panel border-emerald-500/20">
  <div className="text-[10px] uppercase font-bold text-slate-400 mb-1">Accepted</div>
  <div className="text-3xl font-black text-emerald-300">{acceptedApps}</div>
  <div className="text-[11px] text-emerald-400/80 mt-1 font-medium">Selected members</div>
</div>

<div className="p-5 rounded-2xl glass-panel border-rose-500/20">
  <div className="text-[10px] uppercase font-bold text-slate-400 mb-1">Declined</div>
  <div className="text-3xl font-black text-rose-300">{declinedApps}</div>
  <div className="text-[11px] text-rose-400/80 mt-1 font-medium">Not selected</div>
</div>
</div>

{/* Role-Wise Statistics Breakdown for 10 Teams */}
<div className="glass-panel p-6 sm:p-8 rounded-2xl space-y-6">
  <div>
    <h3 className="text-xl font-extrabold text-white">Role-Wise Applicant Statistics</h3>
    <p className="text-xs text-slate-400 mt-1">Breakdown of applicant preference choices across all 10 NeuraMorphix
recruitment teams.</p>
  </div>

  <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
    {roles.map((role) => {
      const count = getRoleApplicantCount(role.role_name);
      const percent = totalApps > 0 ? Math.round((count / (totalApps * 2)) * 100) : 0;
      return (
        <div key={role.role_id} className="p-4 rounded-xl bg-slate-900/80 border border-slate-800 space-y-2">
          <div className="flex items-center justify-between">
            <span className="text-sm font-bold text-slate-100">{role.role_name}</span>
            <span className="text-xs font-mono font-bold text-cyan-300 bg-slate-950 px-2.5 py-1 rounded-lg border
border-slate-800">
              {count} applicants
            </span>
          </div>
          <div className="w-full bg-slate-800 h-2 rounded-full overflow-hidden">
            <div
              className="bg-gradient-to-r from-cyan-500 to-blue-500 h-2 rounded-full transition-all duration-500"
              style={{ width: `${Math.min(100, Math.max(5, percent * 2))}%` }}
            ></div>
          </div>
        </div>
      );
    })}
  </div>
</div>
</div>

{/* TAB 2: APPLICANT MANAGEMENT */}
{activeTab === 'applicants' && (
  <div className="space-y-6 animate-fadeIn">
    {/* Search & Filters */}
    <div className="glass-panel p-4 sm:p-6 rounded-2xl flex flex-col md:flex-row gap-4 justify-between">
      <div className="relative flex-1">
        <Search className="w-4 h-4 text-slate-500 absolute left-3.5 top-3.5" />

```



```

<input
  type="text"
  placeholder="Search applicants by name, ID, email, college, or skills..."
  value={searchQuery}
  onChange={(e) => setSearchQuery(e.target.value)}
  className="w-full pl-10 pr-4 py-2.5 rounded-xl glass-input text-xs"
/>
</div>

<div className="flex flex-wrap gap-3">
  <select
    value={statusFilter}
    onChange={(e) => setStatusFilter(e.target.value)}
    className="px-3 py-2.5 rounded-xl glass-input text-xs bg-slate-900 text-white"
  >
    <option value="ALL">All Statuses</option>
    <option value="Application Received">Application Received</option>
    <option value="Under Review">Under Review</option>
    <option value="Shortlisted">Shortlisted</option>
    <option value="Interview">Interview</option>
    <option value="Information Requested">Information Requested</option>
    <option value="Information Received">Information Received</option>
    <option value="Accepted">Accepted</option>
    <option value="Declined">Declined</option>
  </select>

  <select
    value={roleFilter}
    onChange={(e) => setRoleFilter(e.target.value)}
    className="px-3 py-2.5 rounded-xl glass-input text-xs bg-slate-900 text-white"
  >
    <option value="ALL">All Teams</option>
    {roles.map((r) => (
      <option key={r.role_id} value={r.role_name}>
        {r.role_name}
      </option>
    ))}
  </select>
</div>
</div>

{/* Table of Applicants */}
<div className="glass-panel rounded-2xl overflow-hidden border-slate-800">
  <div className="overflow-x-auto">
    <table className="w-full text-left text-xs">
      <thead className="bg-slate-900/90 text-slate-400 uppercase font-semibold border-b border-slate-800">
        <tr>
          <th className="px-6 py-4">Application ID & Name</th>
          <th className="px-6 py-4">College & Dept</th>
          <th className="px-6 py-4">First Preference</th>
          <th className="px-6 py-4">Second Preference</th>
          <th className="px-6 py-4">Status</th>
          <th className="px-6 py-4 text-right">Actions</th>
        </tr>
      </thead>
      <tbody className="divide-y divide-slate-800/60">
        {filteredApplicants.length === 0 ? (
          <tr>
            <td colspan={6} className="px-6 py-8 text-center text-slate-400">
              No applicants found matching the search criteria.
            </td>
          </tr>
        ) : (
          filteredApplicants.map((app) => (
            <tr key={app.id} className="hover:bg-slate-800/40 transition-colors">
              <td className="px-6 py-4">
                <div className="font-bold text-white text-sm">{app.full_name}</div>
                <div className="font-mono text-cyan-400 font-semibold">{app.application_id}</div>
                <div className="text-[11px] text-slate-400">{app.email}</div>
              </td>
              <td className="px-6 py-4">
                <div className="text-slate-200 font-medium">{app.college}</div>
                <div className="text-[11px] text-slate-400">{app.department} ({app.year})</div>
              </td>
              <td className="px-6 py-4">
                <span className="font-semibold text-cyan-300">{app.first_preference}</span>
              </td>
            </tr>
          ))
        )}
      </tbody>
    </table>
  </div>
</div>

```

```

        </td>
        <td className="px-6 py-4">
          <span className="font-semibold text-amber-300">{app.second_preference}</span>
        </td>
        <td className="px-6 py-4">
          <span
            className={`px-3 py-1 rounded-full text-[11px] font-bold border inline-block ${
              app.status === 'Accepted'
                ? 'bg-emerald-500/20 text-emerald-300 border-emerald-500/40'
                : app.status === 'Declined'
                ? 'bg-rose-500/20 text-rose-300 border-rose-500/40'
                : app.status === 'Interview'
                ? 'bg-amber-500/20 text-amber-300 border-amber-500/40'
                : app.status === 'Information Requested'
                ? 'bg-purple-500/20 text-purple-300 border-purple-500/40'
                : 'bg-cyan-500/20 text-cyan-300 border-cyan-500/40'
            }`}
          >
            {app.status}
          </span>
        </td>
        <td className="px-6 py-4 text-right">
          <button
            type="button"
            onClick={() => setSelectedApplicant(app)}
            className="px-3.5 py-1.5 rounded-lg bg-cyan-500/20 hover:bg-cyan-500 text-cyan-300 hover:text-slate-
950 font-bold border border-cyan-500/40 transition-all flex items-center gap-1 ml-auto"
          >
            <Eye className="w-3.5 h-3.5" />
            Review
          </button>
        </td>
      </tr>
    </tbody>
  </table>
</div>
</div>
</div>
))
}}
</tbody>
</table>
</div>
</div>
</div>
))

{/* APPLICANT DETAIL MODAL */}
{selectedApplicant && (
  <div className="fixed inset-0 z-50 bg-slate-950/80 backdrop-blur-md flex items-center justify-center p-4 overflow-y-
auto">
    <div className="glass-panel w-full max-w-4xl rounded-2xl p-6 sm:p-8 space-y-6 max-h-[90vh] overflow-y-auto border-cyan-
500/30">
      {/* Modal Header */}
      <div className="flex items-start justify-between pb-4 border-b border-slate-800">
        <div>
          <span className="px-3 py-1 rounded-full bg-cyan-950 text-cyan-300 text-[10px] font-mono font-bold uppercase">
            ID: {selectedApplicant.application_id}
          </span>
          <h2 className="text-2xl font-black text-white mt-1">{selectedApplicant.full_name}</h2>
          <p className="text-xs text-slate-400">{selectedApplicant.college} • {selectedApplicant.department}
({selectedApplicant.year})</p>
        </div>

        <button
          type="button"
          onClick={() => setSelectedApplicant(null)}
          className="p-2 rounded-lg bg-slate-800 text-slate-400 hover:text-white"
        >
          ✕
        </button>
      </div>

      {/* Quick Action Bar */}
      <div className="p-4 rounded-xl bg-slate-900 border border-slate-800 flex flex-wrap items-center justify-between gap-
3">
        <div className="text-xs font-semibold text-slate-300">
          Current Status: <span className="text-cyan-300 font-bold">{selectedApplicant.status}</span>
        </div>

        <div className="flex flex-wrap gap-2">

```

```

        <button
          type="button"
          onClick={() => handleShortlist(selectedApplicant)}
          className="px-3 py-1.5 rounded-lg text-xs font-bold bg-blue-500/20 hover:bg-blue-500 text-blue-300 hover:text-slate-950 border border-blue-500/40"
        >
          SHORTLIST
        </button>

        <button
          type="button"
          onClick={() => setShowInterviewModal(true)}
          className="px-3 py-1.5 rounded-lg text-xs font-bold bg-amber-500/20 hover:bg-amber-500 text-amber-300 hover:text-slate-950 border border-amber-500/40"
        >
          REQUEST INTERVIEW
        </button>

        <button
          type="button"
          onClick={() => setShowReqInfoModal(true)}
          className="px-3 py-1.5 rounded-lg text-xs font-bold bg-purple-500/20 hover:bg-purple-500 text-purple-300 hover:text-slate-950 border border-purple-500/40"
        >
          REQUEST INFO
        </button>

        <button
          type="button"
          onClick={() => setShowRoleAssignModal(true)}
          className="px-3 py-1.5 rounded-lg text-xs font-bold bg-slate-800 hover:bg-slate-700 text-slate-200 border border-slate-700"
        >
          CHANGE ROLE
        </button>

        <button
          type="button"
          onClick={() => setShowDeclineModal(true)}
          className="px-3 py-1.5 rounded-lg text-xs font-bold bg-rose-500/20 hover:bg-rose-500 text-rose-300 hover:text-slate-950 border border-rose-500/40"
        >
          DECLINE
        </button>

        <button
          type="button"
          onClick={() => setShowAcceptModal(true)}
          className="px-4 py-1.5 rounded-lg text-xs font-black bg-emerald-500 hover:bg-emerald-400 text-slate-950 shadow-md"
        >
          ACCEPT APPLICANT
        </button>
      </div>
</div>

{/* Application Data Grid */}
<div className="grid grid-cols-1 md:grid-cols-2 gap-6 text-xs">
  <div className="space-y-4">
    <div className="p-4 rounded-xl bg-slate-900/90 border border-slate-800 space-y-2">
      <h4 className="font-bold text-cyan-400 uppercase text-[10px]">Contact Information</h4>
      <div>Email: <strong className="text-white">{selectedApplicant.email}</strong></div>
      <div>Phone: <strong className="text-white">{selectedApplicant.phone}</strong></div>
      <div>Application Date: <strong className="text-white">{new
Date(selectedApplicant.created_at).toLocaleDateString()}</strong></div>
    </div>

    <div className="p-4 rounded-xl bg-slate-900/90 border border-slate-800 space-y-2">
      <h4 className="font-bold text-cyan-400 uppercase text-[10px]">Role Preferences & Assigned Team</h4>
      <div>🟡 First Choice: <strong className="text-cyan-300">{selectedApplicant.first_preference}</strong></div>
      <div>🟢 Second Choice: <strong className="text-amber-300">{selectedApplicant.second_preference}</strong></div>
      <div className="pt-2 border-t border-slate-800">
        Final Assigned Team: <strong className="text-emerald-400">{selectedApplicant.final_assigned_team || 'Not
assigned yet'}</strong>
      </div>
    </div>
  </div>

```

```

<div className="p-4 rounded-xl bg-slate-900/90 border border-slate-800 space-y-2">
  <h4 className="font-bold text-cyan-400 uppercase text-[10px]">Online Links</h4>
  <div className="flex flex-wrap gap-2 pt-1">
    {selectedApplicant.github_url && (
      <a href={selectedApplicant.github_url} target="_blank" rel="noreferrer" className="px-2.5 py-1 rounded bg-slate-800 text-cyan-300 hover:underline flex items-center gap-1">
        GitHub <Externallink className="w-3 h-3" />
      </a>
    )}
    {selectedApplicant.linkedin_url && (
      <a href={selectedApplicant.linkedin_url} target="_blank" rel="noreferrer" className="px-2.5 py-1 rounded bg-slate-800 text-blue-300 hover:underline flex items-center gap-1">
        LinkedIn <Externallink className="w-3 h-3" />
      </a>
    )}
    {selectedApplicant.portfolio_url && (
      <a href={selectedApplicant.portfolio_url} target="_blank" rel="noreferrer" className="px-2.5 py-1 rounded bg-slate-800 text-purple-300 hover:underline flex items-center gap-1">
        Portfolio <Externallink className="w-3 h-3" />
      </a>
    )}
    {selectedApplicant.resume_url && (
      <a href={selectedApplicant.resume_url} target="_blank" rel="noreferrer" className="px-2.5 py-1 rounded bg-slate-800 text-amber-300 hover:underline flex items-center gap-1">
        Resume <Externallink className="w-3 h-3" />
      </a>
    )}
  </div>
</div>
</div>

<div className="space-y-4">
  <div className="p-4 rounded-xl bg-slate-900/90 border border-slate-800 space-y-2">
    <h4 className="font-bold text-cyan-400 uppercase text-[10px]">Skills</h4>
    <div className="flex flex-wrap gap-1.5">
      {selectedApplicant.skills.map((s, i) => (
        <span key={i} className="px-2.5 py-1 rounded bg-slate-800 text-slate-200 border border-slate-700">
          {s}
        </span>
      ))}
    </div>
  </div>
  </div>

  <div className="p-4 rounded-xl bg-slate-900/90 border border-slate-800 space-y-2">
    <h4 className="font-bold text-cyan-400 uppercase text-[10px]">Projects / Experience</h4>
    <p className="text-slate-300 leading-relaxed whitespace-pre-wrap">{selectedApplicant.experience}</p>
  </div>
</div>

{/* Internal Admin Notes Thread */}
<div className="p-4 rounded-xl bg-slate-950 border border-slate-800 space-y-3">
  <h4 className="font-bold text-white text-xs uppercase flex items-center gap-2">
    <MessageSquare className="w-4 h-4 text-cyan-400" />
    Internal Recruiter Notes ({selectedApplicant.admin_notes?.length || 0})
  </h4>

  <div className="space-y-2 max-h-36 overflow-y-auto">
    {selectedApplicant.admin_notes?.length === 0 ? (
      <p className="text-xs text-slate-500 italic">No internal notes added yet.</p>
    ) : (
      selectedApplicant.admin_notes.map((note) => (
        <div key={note.id} className="p-3 rounded-lg bg-slate-900 border border-slate-800 text-xs">
          <div className="flex justify-between text-[10px] text-slate-400 font-semibold mb-1">
            <span>{note.author}</span>
            <span>{new Date(note.created_at).toLocaleString()}</span>
          </div>
          <p className="text-slate-200">{note.text}</p>
        </div>
      ))
    )}
  </div>

  <div className="flex gap-2 pt-2">
    <input

```

```

        type="text"
        placeholder="Type an internal review note..."
        value={noteInput}
        onChange={(e) => setNoteInput(e.target.value)}
        className="flex-1 px-3 py-2 rounded-xl glass-input text-xs"
      />
      <button
        type="button"
        onClick={handleAddNote}
        className="px-4 py-2 rounded-xl bg-cyan-500 text-slate-950 text-xs font-bold"
      >
        ADD NOTE
      </button>
    </div>
  </div>
</div>
</div>
))

{/* ACCEPT CONFIRMATION MODAL */}
{showAcceptModal && selectedApplicant && (
  <div className="fixed inset-0 z-50 bg-slate-950/80 backdrop-blur-md flex items-center justify-center p-4">
    <div className="glass-panel w-full max-w-md rounded-2xl p-6 space-y-4 border-emerald-500/50">
      <h3 className="text-xl font-bold text-white">Accept Applicant Confirmation</h3>
      <p className="text-xs text-slate-300 leading-relaxed">
        Are you sure you want to accept <strong>{selectedApplicant.full_name}</strong> into NeuraMorphix?
      </p>
      <div className="p-3 rounded-xl bg-emerald-950/80 border border-emerald-500/40 text-emerald-200 text-xs">
        This will update their status to <strong>Accepted</strong>, store acceptance metadata, and automatically trigger an
        acceptance notification email.
      </div>

      <div className="flex justify-end gap-3 pt-4">
        <button
          type="button"
          onClick={() => setShowAcceptModal(false)}
          className="px-4 py-2 rounded-xl bg-slate-800 text-slate-300 text-xs font-semibold"
        >
          Cancel
        </button>
        <button
          type="button"
          onClick={handleExecuteAccept}
          className="px-6 py-2 rounded-xl bg-emerald-500 hover:bg-emerald-400 text-slate-950 text-xs font-bold"
        >
          CONFIRM ACCEPTANCE
        </button>
      </div>
    </div>
  </div>
</div>
))

{/* DECLINE CONFIRMATION MODAL */}
{showDeclineModal && selectedApplicant && (
  <div className="fixed inset-0 z-50 bg-slate-950/80 backdrop-blur-md flex items-center justify-center p-4">
    <div className="glass-panel w-full max-w-md rounded-2xl p-6 space-y-4 border-rose-500/50">
      <h3 className="text-xl font-bold text-white">Decline Application Confirmation</h3>
      <p className="text-xs text-slate-300">
        Are you sure you want to decline the application for <strong>{selectedApplicant.full_name}</strong>?
      </p>

      <div>
        <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">Select Decline Reason (Internal)
      </label>

      <select
        value={declineReason}
        onChange={(e) => setDeclineReason(e.target.value as DeclineReasonCategory)}
        className="w-full p-2.5 rounded-xl glass-input text-xs bg-slate-900 text-white"
      >
        <option value="Role capacity reached">Role capacity reached</option>
        <option value="Skills mismatch">Skills mismatch</option>
        <option value="Application incomplete">Application incomplete</option>
        <option value="Selection criteria">Selection criteria</option>
        <option value="Other">Other</option>
      </select>
    </div>
  </div>
</div>
))

```

```

<div>
  <label className="block text-xs font-semibold text-slate-300 uppercase mb-1">Custom Note (Internal Only)</label>
  <textarea
    rows={2}
    placeholder="Optional internal note regarding decline decision..."
    value={declineNote}
    onChange={(e) => setDeclineNote(e.target.value)}
    className="w-full p-2 rounded-xl glass-input text-xs"
  />
</div>

<p className="text-[11px] text-slate-400">
  Note: The applicant will receive a polite email notification. Internal notes will <strong>NOT</strong> be exposed
to the applicant.
</p>

<div className="flex justify-end gap-3 pt-2">
  <button
    type="button"
    onClick={() => setShowDeclineModal(false)}
    className="px-4 py-2 rounded-xl bg-slate-800 text-slate-300 text-xs font-semibold"
  >
    Cancel
  </button>
  <button
    type="button"
    onClick={handleExecuteDecline}
    className="px-6 py-2 rounded-xl bg-rose-500 hover:bg-rose-400 text-slate-950 text-xs font-bold"
  >
    DECLINE APPLICATION
  </button>
</div>
</div>
</div>
)}

{/* REQUEST INFO MODAL */}
{showReqInfoModal && selectedApplicant && (
  <div className="fixed inset-0 z-50 bg-slate-950/80 backdrop-blur-md flex items-center justify-center p-4">
    <div className="glass-panel w-full max-w-md rounded-2xl p-6 space-y-4 border-purple-500/50">
      <h3 className="text-xl font-bold text-white">Request Additional Information</h3>
      <p className="text-xs text-slate-300">
        Enter the specific information or code repository needed from <strong>{selectedApplicant.full_name}</strong>:
      </p>

      <textarea
        rows={3}
        value={reqInfoQuestion}
        onChange={(e) => setReqInfoQuestion(e.target.value)}
        className="w-full p-3 rounded-xl glass-input text-xs"
      />

      <div className="flex justify-end gap-3 pt-2">
        <button
          type="button"
          onClick={() => setShowReqInfoModal(false)}
          className="px-4 py-2 rounded-xl bg-slate-800 text-slate-300 text-xs font-semibold"
        >
          Cancel
        </button>
        <button
          type="button"
          onClick={handleExecuteRequestInfo}
          className="px-6 py-2 rounded-xl bg-purple-500 text-slate-950 text-xs font-bold"
        >
          SEND REQUEST EMAIL
        </button>
      </div>
    </div>
  </div>
)}

{/* REQUEST INTERVIEW MODAL */}
{showInterviewModal && selectedApplicant && (
  <div className="fixed inset-0 z-50 bg-slate-950/80 backdrop-blur-md flex items-center justify-center p-4">

```

```

<div className="glass-panel w-full max-w-md rounded-2xl p-6 space-y-4 border-amber-500/50">
  <h3 className="text-xl font-bold text-white">Request Interview</h3>
  <p className="text-xs text-slate-300">
    Enter interview slot details / Google Meet link for <strong>{selectedApplicant.full_name}</strong>:
  </p>

  <textarea
    rows={3}
    value={interviewDetailsInput}
    onChange={(e) => setInterviewDetailsInput(e.target.value)}
    className="w-full p-3 rounded-xl glass-input text-xs"
  />

  <div className="flex justify-end gap-3 pt-2">
    <button
      type="button"
      onClick={() => setShowInterviewModal(false)}
      className="px-4 py-2 rounded-xl bg-slate-800 text-slate-300 text-xs font-semibold"
    >
      Cancel
    </button>
    <button
      type="button"
      onClick={handleExecuteRequestInterview}
      className="px-6 py-2 rounded-xl bg-amber-500 text-slate-950 text-xs font-bold"
    >
      SEND INTERVIEW INVITATION
    </button>
  </div>
</div>
)}

{/* ROLE ALLOCATION MODAL */}
{showRoleAssignModal && selectedApplicant && (
  <div className="fixed inset-0 z-50 bg-slate-950/80 backdrop-blur-md flex items-center justify-center p-4">
    <div className="glass-panel w-full max-w-md rounded-2xl p-6 space-y-4 border-cyan-500/50">
      <h3 className="text-xl font-bold text-white">Role Allocation / Final Team</h3>
      <p className="text-xs text-slate-300">
        Assign a final team independently of the applicant's preferences.
      </p>

      <div className="text-xs text-slate-400 space-y-1">
        <div> 🏆 1st Choice: <span className="text-cyan-300">{selectedApplicant.first_preference}</span></div>
        <div> 🥈 2nd Choice: <span className="text-amber-300">{selectedApplicant.second_preference}</span></div>
      </div>

      <div>
        <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">Select Final Team Assignment</label>
        <select
          value={assignedRoleChoice || selectedApplicant.first_preference}
          onChange={(e) => setAssignedRoleChoice(e.target.value)}
          className="w-full p-2.5 rounded-xl glass-input text-xs bg-slate-900 text-white"
        >
          {roles.map((r) => (
            <option key={r.role_id} value={r.role_name}>
              {r.role_name}
            </option>
          ))}
        </select>
      </div>

      <div className="flex justify-end gap-3 pt-2">
        <button
          type="button"
          onClick={() => setShowRoleAssignModal(false)}
          className="px-4 py-2 rounded-xl bg-slate-800 text-slate-300 text-xs font-semibold"
        >
          Cancel
        </button>
        <button
          type="button"
          onClick={handleAssignFinalTeam}
          className="px-6 py-2 rounded-xl bg-cyan-500 text-slate-950 text-xs font-bold"
        >
          SAVE ROLE ASSIGNMENT
        </button>
      </div>
    </div>
  </div>
)
}

```

```

    </button>
  </div>
</div>
</div>
))

{/* TAB 3: EMAIL NOTIFICATIONS & TEMPLATE EDITOR */}
{activeTab === 'email_settings' && (
  <div className="space-y-8 animate-fadeIn">
    {/* Toggles */}
    <div className="glass-panel p-6 rounded-2xl space-y-4">
      <h3 className="text-lg font-bold text-white flex items-center gap-2">
        <Mail className="w-5 h-5 text-cyan-400" />
        Automated Email Event Triggers
      </h3>

      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4 text-xs">
        <label className="p-4 rounded-xl bg-slate-900 border border-slate-800 flex items-center justify-between cursor-
pointer">

          <span>☑ Application received email</span>
          <input
            type="checkbox"
            checked={emailSettings.enable_application_received}
            onChange={() => handleToggleEmailSetting('enable_application_received')}
            className="w-4 h-4 rounded text-cyan-500"
          />
        </label>

        <label className="p-4 rounded-xl bg-slate-900 border border-slate-800 flex items-center justify-between cursor-
pointer">

          <span>☑ Shortlist email</span>
          <input
            type="checkbox"
            checked={emailSettings.enable_shortlist}
            onChange={() => handleToggleEmailSetting('enable_shortlist')}
            className="w-4 h-4 rounded text-cyan-500"
          />
        </label>

        <label className="p-4 rounded-xl bg-slate-900 border border-slate-800 flex items-center justify-between cursor-
pointer">

          <span>☑ Interview email</span>
          <input
            type="checkbox"
            checked={emailSettings.enable_interview}
            onChange={() => handleToggleEmailSetting('enable_interview')}
            className="w-4 h-4 rounded text-cyan-500"
          />
        </label>

        <label className="p-4 rounded-xl bg-slate-900 border border-slate-800 flex items-center justify-between cursor-
pointer">

          <span>☑ Information request email</span>
          <input
            type="checkbox"
            checked={emailSettings.enable_info_requested}
            onChange={() => handleToggleEmailSetting('enable_info_requested')}
            className="w-4 h-4 rounded text-cyan-500"
          />
        </label>

        <label className="p-4 rounded-xl bg-slate-900 border border-slate-800 flex items-center justify-between cursor-
pointer">

          <span>☑ Acceptance email</span>
          <input
            type="checkbox"
            checked={emailSettings.enable_acceptance}
            onChange={() => handleToggleEmailSetting('enable_acceptance')}
            className="w-4 h-4 rounded text-cyan-500"
          />
        </label>

        <label className="p-4 rounded-xl bg-slate-900 border border-slate-800 flex items-center justify-between cursor-
pointer">

          <span>☑ Decline email</span>
          <input
            type="checkbox"
            checked={emailSettings.enable_decline}
            onChange={() => handleToggleEmailSetting('enable_decline')}
            className="w-4 h-4 rounded text-cyan-500"
          />
        </label>
      </div>
    </div>
  )}
)

```



```

        type="checkbox"
        checked={emailSettings.enable_decline}
        onChange={() => handleToggleEmailSetting('enable_decline')}
        className="w-4 h-4 rounded text-cyan-500"
      />
    </label>
  </div>
</div>

{/* Template Editor */}
<div className="glass-panel p-6 sm:p-8 rounded-2xl space-y-6">
  <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 border-b border-slate-800 pb-4">
    <div>
      <h3 className="text-xl font-bold text-white">Email Template Editor</h3>
      <p className="text-xs text-slate-400">Customize the subject and content for automated notification emails.</p>
    </div>

    <select
      value={editingTemplateType}
      onChange={(e) => setEditingTemplateType(e.target.value as EmailType)}
      className="px-4 py-2 rounded-xl glass-input text-xs bg-slate-900 text-cyan-300 font-bold"
    >
      <option value="application_received">Application Received Email</option>
      <option value="shortlisted">Shortlisted Email</option>
      <option value="interview">Interview Invitation Email</option>
      <option value="info_requested">Information Request Email</option>
      <option value="accepted">Acceptance Email</option>
      <option value="declined">Decline Email</option>
    </select>
  </div>

  <div className="space-y-4">
    <div>
      <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">Subject Line</label>
      <input
        type="text"
        value={templateSubject}
        onChange={(e) => setTemplateSubject(e.target.value)}
        className="w-full px-4 py-2.5 rounded-xl glass-input text-xs"
      />
    </div>

    <div>
      <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">Body Template (Markdown/Text)
</label>

      <textarea
        rows={10}
        value={templateBody}
        onChange={(e) => setTemplateBody(e.target.value)}
        className="w-full p-4 rounded-xl glass-input text-xs font-mono leading-relaxed"
      />
    </div>

    <div className="p-3 rounded-xl bg-slate-950 border border-slate-800 text-[11px] text-slate-400">
      Available Placeholders: <code>`{{name}}`</code>, <code>`{{application_id}}`</code>, <code>
`{{first_preference}}`</code>, <code>`{{second_preference}}`</code>, <code>`{{final_assigned_team}}`</code>, <code>
`{{requested_info_question}}`</code>, <code>`{{interview_details}}`</code>
    </div>

    <div className="flex justify-end pt-2">
      <button
        type="button"
        onClick={handleSaveEmailTemplate}
        className="px-6 py-2.5 rounded-xl bg-cyan-500 hover:bg-cyan-400 text-slate-950 text-xs font-bold shadow-lg"
      >
        SAVE EMAIL TEMPLATE
      </button>
    </div>
  </div>
</div>
</div>
))

{/* TAB 4: RECRUITMENT CONFIGURATION & DEADLINE CONTROL */}
{activeTab === 'config' && (
  <div className="glass-panel p-6 sm:p-8 rounded-2xl space-y-6 animate-fadeIn">

```

```

<div>
  <h3 className="text-xl font-bold text-white flex items-center gap-2">
    <Sliders className="w-5 h-5 text-cyan-400" />
    Recruitment Period & Deadline Control
  </h3>
  <p className="text-xs text-slate-400 mt-1">Configure recruitment opening and closing dates or manually override
window state.</p>
</div>

<div className="grid grid-cols-1 sm:grid-cols-2 gap-6 text-xs">
  <div>
    <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">Start Date</label>
    <input
      type="date"
      value={config.start_date}
      onChange={(e) => handleSaveConfig({ ...config, start_date: e.target.value })}
      className="w-full px-4 py-2.5 rounded-xl glass-input text-xs"
    />
  </div>

  <div>
    <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">End Date (Deadline)</label>
    <input
      type="date"
      value={config.end_date}
      onChange={(e) => handleSaveConfig({ ...config, end_date: e.target.value })}
      className="w-full px-4 py-2.5 rounded-xl glass-input text-xs"
    />
  </div>
</div>

<div className="p-4 rounded-xl bg-slate-900 border border-slate-800 space-y-3">
  <h4 className="font-bold text-xs text-white uppercase">Manual Override Options</h4>
  <div className="flex flex-wrap gap-3">
    <button
      type="button"
      onClick={() => handleSaveConfig({ ...config, is_manually_open: true })}
      className={`px-4 py-2 rounded-xl text-xs font-bold ${
        config.is_manually_open === true
          ? 'bg-emerald-500 text-slate-950 ring-2 ring-emerald-400'
          : 'bg-slate-800 text-slate-300 hover:bg-slate-700'
      }`}
    >
      FORCE OPEN RECRUITMENT
    </button>

    <button
      type="button"
      onClick={() => handleSaveConfig({ ...config, is_manually_open: false })}
      className={`px-4 py-2 rounded-xl text-xs font-bold ${
        config.is_manually_open === false
          ? 'bg-rose-500 text-slate-950 ring-2 ring-rose-400'
          : 'bg-slate-800 text-slate-300 hover:bg-slate-700'
      }`}
    >
      FORCE CLOSE RECRUITMENT
    </button>

    <button
      type="button"
      onClick={() => handleSaveConfig({ ...config, is_manually_open: null })}
      className={`px-4 py-2 rounded-xl text-xs font-bold ${
        config.is_manually_open === null
          ? 'bg-cyan-500 text-slate-950 ring-2 ring-cyan-400'
          : 'bg-slate-800 text-slate-300 hover:bg-slate-700'
      }`}
    >
      USE AUTOMATIC DATES
    </button>
  </div>
</div>
</div>
  </div>
)
</div>

```

```
);  
};
```

Candidate Application Form Component

[src/components/ApplicationForm.tsx](#)

Multi-step applicant registration form with Indian Colleges autocomplete, 10-digit phone validator, and confetti celebration.

```
import React, { useState, useRef, useEffect } from 'react';  
import type { Applicant, Role } from '../types/recruitment';  
import { DatabaseService } from '../services/db';  
import { EmailService } from '../services/email';  
import { BackendApiService } from '../services/api';  
import { INDIAN_COLLEGES } from '../data/indianColleges';  
import confetti from 'canvas-confetti';  
  
import {  
  User,  
  Mail,  
  Phone,  
  GraduationCap,  
  Code,  
  FileCheck,  
  ArrowLeft,  
  ArrowRight,  
  CheckCircle,  
  CheckCircle2,  
  ShieldCheck,  
  AlertTriangle,  
  RotateCcw,  
  Search,  
  Camera,  
  XCircle,  
} from 'lucide-react';  
  
interface ApplicationFormProps {  
  firstChoice: string;  
  secondChoice: string | null;  
  roles: Role[];  
  onChangePreferences: () => void;  
  onApplicationSubmitted: (applicant: Applicant) => void;  
  onTrackStatusDirectly?: (appId: string) => void;  
}  
  
export const ApplicationForm: React.FC<ApplicationFormProps> = ({  
  firstChoice,  
  secondChoice,  
  roles,  
  onChangePreferences,  
  onApplicationSubmitted,  
  onTrackStatusDirectly,  
}) => {  
  const [step, setStep] = useState<  
    'details' | 'skills_experience' | 'review' | 'submitted'  
  >('details');  
  
  // =====  
  // FORM FIELDS  
  // =====  
  
  const [fullName, setFullName] = useState('');  
  const [email, setEmail] = useState('');  
  const [phone, setPhone] = useState('');  
  const [college, setCollege] = useState('');  
  const [department, setDepartment] = useState('');  
  const [year, setYear] = useState('3rd Year');  
  
  const [resumeUrl, setResumeUrl] = useState('');  
  const [githubUrl, setGithubUrl] = useState('');  
  const [linkedinUrl, setLinkedinUrl] = useState('');  
  const [portfolioUrl, setPortfolioUrl] = useState('');  
  
  // =====
```

```

// PHONE VALIDATION
// =====

const phoneDigits = phone.replace(/\D/g, '');
const isPhoneValid = phoneDigits.length === 10;
const phoneHasInput = phone.trim().length > 0;

// =====
// COLLEGE AUTOCOMPLETE
// =====

const [collegeQuery, setCollegeQuery] = useState('');
const [showCollegeSuggestions, setShowCollegeSuggestions] =
  useState(false);

const collegeRef = useRef<HTMLDivElement>(null);

const collegeSuggestions =
  collegeQuery.trim().length >= 2
    ? INDIAN_COLLEGES.filter((c) =>
        c
          .toLowerCase()
          .includes(collegeQuery.toLowerCase())
        ).slice(0, 8)
    : [];

const isCollegeVerified =
  INDIAN_COLLEGES.includes(college) &&
  college.trim().length > 0;

useEffect(() => {
  const handleOutside = (e: MouseEvent) => {
    if (
      collegeRef.current &&
      !collegeRef.current.contains(
        e.target as Node
      )
    ) {
      setShowCollegeSuggestions(false);
    }
  };

  document.addEventListener(
    'mousedown',
    handleOutside
  );

  return () => {
    document.removeEventListener(
      'mousedown',
      handleOutside
    );
  };
}, []);

const handleCollegeSelect = (name: string) => {
  setCollege(name);
  setCollegeQuery(name);
  setShowCollegeSuggestions(false);
};

// =====
// SKILLS & EXPERIENCE
// =====

const [selectedSkills, setSelectedSkills] =
  useState<string[]>([]);

const [customSkillInput, setCustomSkillInput] =
  useState('');

const [experience, setExperience] =
  useState('');

// =====
// REVIEW & SUBMISSION

```

```

// =====

const [confirmed, setConfirmed] =
  useState(false);

const [errorMsg, setErrorMsg] =
  useState<string | null>(null);

const [submittedApplicant, setSubmittedApplicant] =
  useState<Applicant | null>(null);

const [isSubmitting, setIsSubmitting] =
  useState(false);

const [emailStatus, setEmailStatus] = useState<
  'idle' | 'sending' | 'sent' | 'failed'
>('idle');

// =====
// ROLE SKILLS
// =====

const firstRoleObj = roles.find(
  (r) => r.role_name === firstChoice
);

const secondRoleObj = roles.find(
  (r) => r.role_name === secondChoice
);

const suggestedSkills = Array.from(
  new Set([
    ...(firstRoleObj?.skills || []),
    ...(secondRoleObj?.skills || []),
  ])
);

// =====
// SKILL FUNCTIONS
// =====

const toggleSkill = (skill: string) => {
  if (selectedSkills.includes(skill)) {
    setSelectedSkills(
      selectedSkills.filter(
        (s) => s !== skill
      )
    );
  } else {
    setSelectedSkills([
      ...selectedSkills,
      skill,
    ]);
  }
};

const addCustomSkill = () => {
  const skill = customSkillInput.trim();

  if (
    skill &&
    !selectedSkills.includes(skill)
  ) {
    setSelectedSkills([
      ...selectedSkills,
      skill,
    ]);

    setCustomSkillInput('');
  }
};

// =====
// STEP 1 → STEP 2
// =====

```

```

const handleNextFromDetails = (
  e: React.FormEvent
) => {
  e.preventDefault();

  setErrorMsg(null);

  if (
    !fullName.trim() ||
    !email.trim() ||
    !phone.trim() ||
    !college.trim() ||
    !department.trim()
  ) {
    setErrorMsg(
      'Please fill in all required personal information fields.'
    );

    return;
  }

  if (!isPhoneValid) {
    setErrorMsg(
      'Please enter a valid 10-digit phone number.'
    );

    return;
  }

  const windowCheck =
    DatabaseService.isRecruitmentOpen();

  if (!windowCheck.isOpen) {
    setErrorMsg(windowCheck.message);
    return;
  }

  setStep('skills_experience');
};

// =====
// STEP 2 → STEP 3
// =====

const handleNextFromSkills = (
  e: React.FormEvent
) => {
  e.preventDefault();

  setErrorMsg(null);

  if (selectedSkills.length === 0) {
    setErrorMsg(
      'Please select or add at least one relevant skill.'
    );

    return;
  }

  if (!experience.trim()) {
    setErrorMsg(
      'Please provide a brief description of your previous projects or experience.'
    );

    return;
  }

  setStep('review');
};

// =====
// SUBMIT APPLICATION
// FRONTEND → BACKEND EMAIL API
// =====

const handleSubmitApplication =

```

```

async () => {
  setErrorMsg(null);

  if (!confirmed) {
    setErrorMsg(
      'You must check the confirmation box before submitting your application.'
    );

    return;
  }

  const windowCheck =
    DatabaseService.isRecruitmentOpen();

  if (!windowCheck.isOpen) {
    setErrorMsg(windowCheck.message);
    return;
  }

  setIsSubmitting(true);
  setEmailStatus('sending');

  try {
    // -----
    // Generate application ID
    // -----

    const randomNum = Math.floor(
      10000 + Math.random() * 90000
    );

    const appId =
      `NM-2026-${randomNum}`;

    // -----
    // Create applicant object
    // -----

    const newApplicant: Applicant = {
      id: `app-${Date.now()}`,

      application_id: appId,

      full_name: fullName.trim(),

      email: email.trim(),

      phone: phone.trim(),

      college: college.trim(),

      department: department.trim(),

      year,

      skills: selectedSkills,

      experience: experience.trim(),

      first_preference: firstChoice,

      second_preference:
        secondChoice ||
        'None (Optional)',

      final_assigned_team: null,

      status:
        'Application Received',

      resume_url:
        resumeUrl.trim(),

      github_url:
        githubUrl.trim(),

```

```

    linkedin_url:
      linkedinUrl.trim(),

    portfolio_url:
      portfolioUrl.trim(),

    admin_notes: [],

    decline_reason: null,

    decline_note: null,

    requested_info_question:
      null,

    requested_info_response:
      null,

    interview_details:
      null,

    created_at:
      new Date().toISOString(),

    updated_at:
      new Date().toISOString(),

    reviewed_at: null,

    accepted_at: null,

    declined_at: null,
  };

  // -----
  // SAVE APPLICATION
  // -----

  DatabaseService.addApplicant(
    newApplicant
  );

  // Sync to Spring Boot MySQL backend asynchronously
  BackendApiService.syncApplicant(
    newApplicant
  );

  // -----
  // SEND EMAIL
  //
  // React frontend
  //   ↓
  // EmailService
  //   ↓
  // /api/send-email
  //   ↓
  // Nodemailer
  //   ↓
  // Gmail SMTP
  // -----

  const emailResult =
    await EmailService.sendEmail(
      'application_received',
      newApplicant
    );

  if (emailResult.success) {
    setEmailStatus('sent');
  } else {
    setEmailStatus('failed');

    console.warn(
      'Application saved, but email could not be sent:',
      emailResult.message
    );
  }

```



```

    }

    // -----
    // CONFETTI
    // -----

    try {
      confetti({
        particleCount: 120,
        spread: 80,
        origin: {
          y: 0.6,
        },
      });
    } catch (e) {
      console.log(
        'Confetti triggered',
        e
      );
    }

    // -----
    // SHOW SUCCESS SCREEN
    // -----

    setSubmittedApplicant(
      newApplicant
    );

    setStep('submitted');

    onApplicationSubmitted(
      newApplicant
    );
  } catch (err) {
    console.error(
      'Submission error:',
      err
    );

    setEmailStatus('failed');

    setErrorMsg(
      'An error occurred during submission. Please try again.'
    );
  } finally {
    setIsSubmitting(false);
  }
};

// =====
// UI
// =====

return (
  <div className="max-w-4xl mx-auto py-8 px-4 sm:px-6">

    {/* =====
      PROGRESS HEADER
    ===== */}

    <div className="mb-8 glass-panel rounded-2xl p-4 flex items-center justify-between">

      <div className="flex items-center gap-2">

        <span className="w-8 h-8 rounded-full bg-cyan-500/20 text-cyan-400 flex items-center justify-center font-bold text-sm">

          {step === 'details'
            ? '1'
            : step ===
              'skills_experience'
            ? '2'
            : step === 'review'
            ? '3'
            : '✓'}

        

```



```

        setEmail(
            e.target.value
        )
    }
    className="w-full pl-10 pr-4 py-2.5 rounded-xl glass-input text-sm"
  />

</div>

</div>

{/* PHONE */}

<div>

  <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">
    Phone Number{' '}
    <span className="text-rose-400">
      *
    </span>
  </label>

  <div className="relative">

    <Phone className="w-4 h-4 text-slate-500 absolute left-3.5 top-3.5" />

    <input
      type="tel"
      required
      maxLength={10}
      placeholder="10-digit number e.g. 9876543210"
      value={phone}
      onChange={(e) =>
        setPhone(
          e.target.value
            .replace(
              /[^0-9]/g,
              ''
            )
            .slice(0, 10)
        )
      }
      className={`w-full pl-10 py-2.5 rounded-xl glass-input text-sm transition-all ${
        phoneHasInput
          ? isPhoneValid
            ? 'pr-10 border border-emerald-500/60'
            : 'pr-10 border border-rose-500/60'
          : 'pr-4'
        }`}
    />

    {phoneHasInput && (
      <div className="absolute right-3 top-3">

        {isPhoneValid ? (
          <CheckCircle2 className="w-4 h-4 text-emerald-400" />
        ) : (
          <XCircle className="w-4 h-4 text-rose-400" />
        )}

      </div>
    )}

  </div>

</div>

{/* COLLEGE */}

<div ref={collegeRef}>

  <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">
    College / Institution{' '}
    <span className="text-rose-400">
      *
    </span>
  </label>

```

```

</label>

<div className="relative">

  <GraduationCap className="w-4 h-4 text-slate-500 absolute left-3.5 top-3.5 z-10" />

  <input
    type="text"
    required
    autoComplete="off"
    placeholder="Type to search college..."
    value={collegeQuery}
    onFocus={() =>
      setShowCollegeSuggestions(
        true
      )
    }
    onChange={(e) => {
      setCollegeQuery(
        e.target.value
      );

      setCollege(
        e.target.value
      );

      setShowCollegeSuggestions(
        true
      );
    }}
    className={`w-full pl-10 py-2.5 rounded-xl glass-input text-sm transition-all ${
      college.trim()
        .length > 0
        ? isCollegeVerified
          ? 'pr-10 border border-emerald-500/60'
          : 'pr-4'
        : 'pr-4'
    }`}
  />

  {isCollegeVerified && (
    <div className="absolute right-3 top-3">
      <CheckCircle2 className="w-4 h-4 text-emerald-400" />
    </div>
  )}

  {showCollegeSuggestions &&
    collegeSuggestions.length >
    0 && (
      <div className="absolute z-50 top-full left-0 right-0 mt-1 bg-slate-900 border border-slate-700 rounded-xl
shadow-2xl overflow-hidden max-h-56 overflow-y-auto">

        {collegeSuggestions.map(
          (name, i) => (
            <button
              key={i}
              type="button"
              onMouseDown={() =>
                handleCollegeSelect(
                  name
                )
              }
              className="w-full text-left px-4 py-2.5 text-xs text-slate-200 hover:bg-cyan-900/60 hover:text-cyan-
300 flex items-center gap-2 border-b border-slate-800 last:border-0 transition-colors"
            >
              <GraduationCap className="w-3.5 h-3.5 text-slate-500 shrink-0" />

              {name}
            </button>
          )
        )}

      </div>
    )}

</div>

```

```

    {college.trim().length >
      0 && (
        <div
          className={`mt-1 text-[11px] font-medium ${
            isCollegeVerified
              ? 'text-emerald-400'
              : 'text-slate-400'
            }`}
        >
          {isCollegeVerified
            ? '✓ College verified from list'
            : 'Not from list – you can still type your college name'}
          </div>
        )}
  </div>

  {/* DEPARTMENT */}

  <div>

    <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">
      Department / Major{' '}
      <span className="text-rose-400">
        *
      </span>
    </label>

    <input
      type="text"
      required
      placeholder="e.g. Computer Science / Electronics"
      value={department}
      onChange={(e) =>
        setDepartment(
          e.target.value
        )
      }
      className="w-full px-4 py-2.5 rounded-xl glass-input text-sm"
    />

  </div>

  {/* YEAR */}

  <div>

    <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">
      Academic Year{' '}
      <span className="text-rose-400">
        *
      </span>
    </label>

    <select
      value={year}
      onChange={(e) =>
        setYear(
          e.target.value
        )
      }
      className="w-full px-4 py-2.5 rounded-xl glass-input text-sm bg-slate-900 text-white"
    >
      <option value="1st Year">
        1st Year
      </option>

      <option value="2nd Year">
        2nd Year
      </option>

      <option value="3rd Year">
        3rd Year
      </option>
    </select>
  </div>

```

```

        <option value="4th Year">
          4th Year
        </option>

        <option value="Postgraduate / PhD">
          Postgraduate / PhD
        </option>
      </select>

    </div>

  </div>

  { /* ONLINE PROFILES */ }

  <div className="pt-4 border-t border-slate-800 space-y-4">

    <h3 className="text-sm font-semibold text-slate-300">
      Online Profiles & Portfolio Links (Optional)
    </h3>

    <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">

      <input
        type="url"
        placeholder="GitHub Profile URL"
        value={githubUrl}
        onChange={(e) =>
          setGithubUrl(
            e.target.value
          )
        }
        className="w-full px-4 py-2 rounded-xl glass-input text-xs"
      />

      <input
        type="url"
        placeholder="LinkedIn Profile URL"
        value={linkedinUrl}
        onChange={(e) =>
          setLinkedInUrl(
            e.target.value
          )
        }
        className="w-full px-4 py-2 rounded-xl glass-input text-xs"
      />

      <input
        type="url"
        placeholder="Portfolio / Personal Website URL"
        value={portfolioUrl}
        onChange={(e) =>
          setPortfolioUrl(
            e.target.value
          )
        }
        className="w-full px-4 py-2 rounded-xl glass-input text-xs"
      />

      <input
        type="url"
        placeholder="Resume Drive/PDF Link"
        value={resumeUrl}
        onChange={(e) =>
          setResumeUrl(
            e.target.value
          )
        }
        className="w-full px-4 py-2 rounded-xl glass-input text-xs"
      />

    </div>

  </div>

  { /* NAVIGATION */ }

```

```

    <div className="flex justify-between pt-6 border-t border-slate-800">

      <button
        type="button"
        onClick={
          onChangePreferences
        }
        className="px-5 py-2.5 rounded-xl text-xs font-semibold bg-slate-800 text-slate-300 hover:bg-slate-700 flex items-
center gap-2">
        >
        <ArrowLeft className="w-4 h-4" />

        Back to Role Selection
      </button>

      <button
        type="submit"
        className="px-6 py-2.5 rounded-xl text-xs font-bold bg-cyan-500 hover:bg-cyan-400 text-slate-950 flex items-center
gap-2 shadow-lg shadow-cyan-500/20">
        >
        Continue to Skills & Experience

        <ArrowRight className="w-4 h-4" />
      </button>

    </div>

  </form>
)}

{/* =====
STEP 2
===== */}

{step === 'skills_experience' && (
  <form
    onSubmit={
      handleNextFromSkills
    }
    className="glass-panel p-6 sm:p-8 rounded-2xl space-y-6">
    >

    <div className="border-b border-slate-800 pb-4">

      <h2 className="text-2xl font-bold text-white flex items-center gap-2">

        <Code className="w-6 h-6 text-cyan-400" />

        Skills & Experience

      </h2>

      <p className="text-sm text-slate-400">
        Highlight your expertise relevant to your selected role preferences.
      </p>

    </div>

    <div>

      <label className="block text-xs font-semibold text-slate-300 uppercase mb-3">
        Select Relevant Skills{' '}
        <span className="text-rose-400">
          *
        </span>
      </label>

      <div className="flex flex-wrap gap-2 mb-4">

        {suggestedSkills.map(
          (skill, idx) => {
            const active =
              selectedSkills.includes(

```



```

        skill
    );

    return (
    <button
        key={idx}
        type="button"
        onClick={() =>
            toggleSkill(
                skill
            )
        }
        className={`px-3 py-1.5 rounded-xl text-xs font-medium transition-all ${
            active
                ? 'bg-cyan-500 text-slate-950 font-bold border border-cyan-400 shadow-md'
                : 'bg-slate-800/90 text-slate-300 hover:bg-slate-700 border border-slate-700'
            }`}
    >
        {active
            ? '✓ '
            : '+ '}

        {skill}
    </button>
    );
}
})

</div>

{/* CUSTOM SKILL */}

<div className="flex gap-2 max-w-md">

    <input
        type="text"
        placeholder="Add custom skill (e.g. OpenCV, Docker)"
        value={customSkillInput}
        onChange={(e) =>
            setCustomSkillInput(
                e.target.value
            )
        }
        onKeyDown={(e) => {
            if (
                e.key === 'Enter'
            ) {
                e.preventDefault();
                addCustomSkill();
            }
        }}
        className="px-4 py-2 rounded-xl glass-input text-xs flex-1"
    />

    <button
        type="button"
        onClick={
            addCustomSkill
        }
        className="px-4 py-2 rounded-xl bg-slate-800 text-slate-200 text-xs font-semibold border border-slate-700
        hover:bg-slate-700"
    >
        Add Skill
    </button>

</div>

</div>

{/* EXPERIENCE */}

<div>

    <label className="block text-xs font-semibold text-slate-300 uppercase mb-2">
        Previous Projects / Experience{' '}
    <span className="text-rose-400">

```

```

        *
      </span>
    </label>

    <textarea
      required
      rows={5}
      placeholder="Describe your previous projects, team experience, code repositories, design portfolios, or research
contributions..."
      value={experience}
      onChange={(e) =>
        setExperience(
          e.target.value
        )
      }
      className="w-full p-4 rounded-xl glass-input text-sm leading-relaxed"
    />

  </div>

  { /* NAVIGATION */ }

  <div className="flex justify-between pt-6 border-t border-slate-800">

    <button
      type="button"
      onClick={() =>
        setStep('details')
      }
      className="px-5 py-2.5 rounded-xl text-xs font-semibold bg-slate-800 text-slate-300 hover:bg-slate-700 flex items-
center gap-2"
    >
      <ArrowLeft className="w-4 h-4" />

      Back to Personal Info
    </button>

    <button
      type="submit"
      className="px-6 py-2.5 rounded-xl text-xs font-bold bg-cyan-500 hover:bg-cyan-400 text-slate-950 flex items-center
gap-2 shadow-lg shadow-cyan-500/20"
    >
      Proceed to Final Review

      <ArrowRight className="w-4 h-4" />
    </button>

  </div>

</form>
)}

{ /* =====
STEP 3 - REVIEW
===== */ }

{step === 'review' && (
  <div className="glass-panel p-6 sm:p-8 rounded-2xl space-y-8">

    <div className="border-b border-slate-800 pb-4 text-center">

      <span className="inline-flex items-center gap-1.5 px-3 py-1 rounded-full bg-cyan-950 text-cyan-300 text-xs font-bold
uppercase mb-2">

        <FileCheck className="w-3.5 h-3.5" />

        Final Step

      </span>

      <h2 className="text-3xl font-extrabold text-white tracking-tight">
        REVIEW YOUR APPLICATION
      </h2>

      <p className="text-slate-400 text-xs mt-1">
        Please verify all details before submitting your application to NeuraMorphix.

```

```

    </p>

  </div>

  { /* ROLE PREFERENCES */ }

  <div className="p-5 rounded-xl bg-slate-900/90 border border-slate-800 space-y-3">

    <div className="flex items-center justify-between">

      <h3 className="text-xs font-bold text-cyan-400 uppercase tracking-wider">
        Role Preferences
      </h3>

      <button
        type="button"
        onClick={
          onChangePreferences
        }
        className="text-xs text-slate-400 hover:text-cyan-300 font-medium underline"
      >
        Change Preferences
      </button>

    </div>

    <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">

      <div className="p-3 rounded-lg bg-slate-950 border border-slate-800">

        <div className="text-[10px] text-slate-400 font-bold uppercase">
          🏆 First Choice
        </div>

        <div className="text-sm font-extrabold text-cyan-300">
          {firstChoice}
        </div>

      </div>

      <div className="p-3 rounded-lg bg-slate-950 border border-slate-800">

        <div className="text-[10px] text-slate-400 font-bold uppercase">
          🥈 Second Choice (Optional)
        </div>

        <div className="text-sm font-extrabold text-amber-300">
          {secondChoice || (
            <span className="text-slate-500 italic font-normal">
              None selected (Optional)
            </span>
          )}
        </div>

      </div>

    </div>

  </div>

  { /* PERSONAL SUMMARY */ }

  <div className="p-5 rounded-xl bg-slate-900/90 border border-slate-800 space-y-3">

    <div className="flex items-center justify-between">

      <h3 className="text-xs font-bold text-cyan-400 uppercase tracking-wider">
        Personal Information
      </h3>

      <button
        type="button"
        onClick={() =>
          setStep('details')
        }

```

```

    }
    className="text-xs text-slate-400 hover:text-cyan-300 font-medium underline"
  >
    Edit
  </button>

</div>

<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4 text-xs">

  <div>
    <span className="text-slate-400 block">
      Name:
    </span>

    <span className="font-semibold text-slate-200">
      {fullName}
    </span>
  </div>

  <div>
    <span className="text-slate-400 block">
      Email:
    </span>

    <span className="font-semibold text-slate-200">
      {email}
    </span>
  </div>

  <div>
    <span className="text-slate-400 block">
      Phone:
    </span>

    <span className="font-semibold text-slate-200">
      {phone}
    </span>
  </div>

  <div>
    <span className="text-slate-400 block">
      College:
    </span>

    <span className="font-semibold text-slate-200">
      {college}
    </span>
  </div>

  <div>
    <span className="text-slate-400 block">
      Department:
    </span>

    <span className="font-semibold text-slate-200">
      {department}
    </span>
  </div>

  <div>
    <span className="text-slate-400 block">
      Year:
    </span>

    <span className="font-semibold text-slate-200">
      {year}
    </span>
  </div>

</div>

</div>

{/* SKILLS SUMMARY */}

```

```

<div className="p-5 rounded-xl bg-slate-900/90 border border-slate-800 space-y-3">

  <div className="flex items-center justify-between">

    <h3 className="text-xs font-bold text-cyan-400 uppercase tracking-wider">
      Skills & Experience
    </h3>

    <button
      type="button"
      onClick={() =>
        setStep(
          'skills_experience'
        )
      }
      className="text-xs text-slate-400 hover:text-cyan-300 font-medium underline"
    >
      Edit
    </button>

  </div>

  <div>

    <span className="text-slate-400 text-xs block mb-1.5">
      Selected Skills:
    </span>

    <div className="flex flex-wrap gap-1.5">

      {selectedSkills.map(
        (sk, i) => (
          <span
            key={i}
            className="px-2.5 py-1 rounded-md bg-slate-800 text-slate-200 text-xs font-medium border border-slate-700"
          >
            {sk}
          </span>
        )
      )}

    </div>

  </div>

  <div className="pt-2">

    <span className="text-slate-400 text-xs block mb-1">
      Previous Projects / Experience:
    </span>

    <p className="text-xs text-slate-300 whitespace-pre-wrap leading-relaxed bg-slate-950 p-3 rounded-lg border border-slate-800">
      {experience}
    </p>

  </div>

</div>

{/* CONFIRMATION */}

<div className="p-4 rounded-xl bg-slate-950 border border-slate-800 flex items-start gap-3">

  <input
    type="checkbox"
    id="confirm-check"
    checked={confirmed}
    onChange={(e) =>
      setConfirmed(
        e.target.checked
      )
    }
    className="mt-1 w-4 h-4 rounded text-cyan-500 focus:ring-cyan-400 bg-slate-900 border-slate-700 cursor-pointer"
  />

```

```

        <label
          htmlFor="confirm-check"
          className="text-xs text-slate-300 leading-relaxed cursor-pointer"
        >
          I confirm that the information provided is accurate and I understand that my role preferences are subject to the
Neuramorphix selection process.
        </label>

      </div>

      { /* SUBMIT */ }

      <div className="flex justify-between pt-4 border-t border-slate-800">

        <button
          type="button"
          onClick={() =>
            setStep(
              'skills_experience'
            )
          }
          className="px-5 py-2.5 rounded-xl text-xs font-semibold bg-slate-800 text-slate-300 hover:bg-slate-700 flex items-
center gap-2"
        >
          <ArrowLeft className="w-4 h-4" />

          Back
        </button>

        <button
          type="button"
          disabled={
            !confirmed ||
            isSubmitting
          }
          onClick={
            handleSubmitApplication
          }
          className={`px-8 py-3 rounded-xl text-sm font-extrabold flex items-center gap-2 shadow-xl transition-all ${
            confirmed &&
            !isSubmitting
              ? 'bg-gradient-to-r from-cyan-400 to-blue-600 hover:from-cyan-300 hover:to-blue-500 text-slate-950 shadow-cyan-
500/25 ring-2 ring-cyan-300 cursor-pointer'
              : 'bg-slate-800 text-slate-500 border border-slate-700 cursor-not-allowed opacity-60'
            }`}
        >

          {isSubmitting ? (
            <>
              Sending Application...
            </>
          ) : (
            <>
              <ShieldCheck className="w-4 h-4" />

              SUBMIT APPLICATION
            </>
          )}

        </button>

      </div>

    </div>
  )}

  { /* =====
STEP 4 - SUCCESS
===== */ }

  {step === 'submitted' &&
    submittedApplicant && (
      <div className="glass-panel p-8 rounded-2xl text-center space-y-6 animate-fadeIn">

        { /* SUCCESS ICON */ }

```

```

<div className="w-16 h-16 rounded-full bg-emerald-500/20 text-emerald-400 border border-emerald-500/50 flex items-center justify-center mx-auto shadow-[0_0_30px_rgba(16,185,129,0.3)]">

    <CheckCircle className="w-10 h-10" />

</div>

{/* TITLE */}

<div className="space-y-2">

    <span className="px-3 py-1 rounded-full bg-emerald-950 text-emerald-300 text-xs font-bold uppercase border border-emerald-500/30">
        Application Status: Received
    </span>

    <h2 className="text-3xl font-extrabold text-white">
        Application Received!
    </h2>

    <p className="text-slate-300 text-sm max-w-lg mx-auto">

        Thank you,{ ' '}

        <strong className="text-white">
            {
                submittedApplicant.full_name
            }
        </strong>
        !

        Your registration for the NeuraMorphix 2026 Team Recruitment has been successfully received.

    </p>

</div>

{/* APPLICATION ID */}

<div className="p-6 rounded-2xl bg-slate-950 border border-cyan-500/30 max-w-md mx-auto shadow-2xl space-y-3">

    <div className="text-xs text-slate-400 uppercase font-semibold">
        Application Number
    </div>

    <div className="text-2xl font-mono font-black text-cyan-300 tracking-wider">
        {
            submittedApplicant.application_id
        }
    </div>

    <div className="text-[11px] text-slate-400">

        Registered Name:{ ' '}

        <strong className="text-slate-200">
            {
                submittedApplicant.full_name
            }
        </strong>

    </div>

    <div className="pt-2.5 border-t border-slate-800 text-amber-300 text-xs font-semibold flex items-center justify-center gap-2 bg-amber-950/40 py-2.5 px-4 rounded-xl border border-amber-500/30">

        <Camera className="w-4 h-4 text-amber-400 shrink-0" />

        <span>
            Note: Remember your application ID or take a screenshot
        </span>

    </div>

</div>

```

```

    { /* EMAIL STATUS */ }

    <div className="flex justify-center pt-2">

        {emailStatus ===
        'sent' && (
            <span className="px-5 py-2.5 rounded-2xl bg-emerald-950/90 text-emerald-300 border border-emerald-500/40 text-xs
font-black uppercase tracking-wider flex items-center gap-2 shadow-lg shadow-emerald-500/10">

                <CheckCircle className="w-4 h-4 text-emerald-400" />

                Email Sent Successfully

            </span>
        )}

        {emailStatus ===
        'failed' && (
            <span className="px-5 py-2.5 rounded-2xl bg-rose-950/90 text-rose-300 border border-rose-500/40 text-xs font-
black uppercase tracking-wider flex items-center gap-2">

                <XCircle className="w-4 h-4 text-rose-400" />

                Application Saved – Email Failed

            </span>
        )}

    </div>

    { /* BUTTONS */ }

    <div className="pt-4 flex flex-wrap justify-center gap-4">

        {onTrackStatusDirectly && (
            <button
                type="button"
                onClick={() =>
                    onTrackStatusDirectly(
                        submittedApplicant.application_id
                    )
                }
                className="px-6 py-3 rounded-xl bg-gradient-to-r from-cyan-500 to-blue-600 hover:from-cyan-400 hover:to-blue-
500 text-slate-950 text-xs font-black cursor-pointer shadow-xl flex items-center gap-2 transition-all hover:scale-105"
            >

                <Search className="w-4 h-4 text-slate-950" />

                <span>
                    Track Application Status
                    →
                </span>

            </button>
        )}

        <button
            type="button"
            onClick={() => {
                setStep('details');
                setSubmittedApplicant(null);
                setConfirmed(false);
                setEmailStatus('idle');

                setFullName('');
                setEmail('');
                setPhone('');
                setCollege('');
                setCollegeQuery('');
                setDepartment('');
                setYear('3rd Year');

                setResumeUrl('');
                setGithubUrl('');
                setLinkedinUrl('');
                setPortfolioUrl('');
            }}
        >
    
```



```

        setSelectedSkills([]);
        setCustomSkillInput('');
        setExperience('');
        setErrorMsg(null);
    })
    className="px-5 py-3 rounded-xl bg-slate-800 hover:bg-slate-700 text-slate-300 text-xs font-bold border border-
    slate-700 cursor-pointer transition-all"
    >
        Submit Another Application
    </button>

</div>

</div>
)}

</div>
);
};

```

Role Selection Component

[src/components/RoleSelectionSection.tsx](#)

Interactive role selector displaying all 10 recruitment teams with primary and optional secondary preference picking.

```

import React, { useState } from 'react';
import type { Role } from '../types/recruitment';
import { RoleCard } from './RoleCard';
import { Sparkles, ArrowRight, RefreshCw, CheckCircle2, AlertCircle } from 'lucide-react';

interface RoleSelectionSectionProps {
  roles: Role[];
  firstChoice: string | null;
  secondChoice: string | null;
  onSelectFirstChoice: (roleName: string) => void;
  onSelectSecondChoice: (roleName: string) => void;
  onClearPreferences: () => void;
  onProceedToForm: () => void;
}

export const RoleSelectionSection: React.FC<RoleSelectionSectionProps> = ({
  roles,
  firstChoice,
  secondChoice,
  onSelectFirstChoice,
  onSelectSecondChoice,
  onClearPreferences,
  onProceedToForm,
}) => {
  const [warningMsg, setWarningMsg] = useState<string | null>(null);

  const handleSelectFirst = (roleName: string) => {
    setWarningMsg(null);
    if (secondChoice === roleName) {
      setWarningMsg(`Swapped choices! "${roleName}" is now your 1st Preference.`);
    }
    onSelectFirstChoice(roleName);
  };

  const handleSelectSecond = (roleName: string) => {
    setWarningMsg(null);
    if (firstChoice === roleName) {
      setWarningMsg(`Swapped choices! "${roleName}" is now your 2nd Preference.`);
    }
    onSelectSecondChoice(roleName);
  };

  return (
    <section id="roles-section" className="py-12 px-4 sm:px-6 lg:px-8 max-w-7xl mx-auto">
      <div className="text-center max-w-3xl mx-auto mb-10">
        <div className="inline-flex items-center gap-2 px-4 py-1.5 rounded-full bg-cyan-950/80 border border-cyan-500/30 text-

```

```

cyan-300 text-xs font-semibold uppercase tracking-wider mb-4 shadow-inner">
  <Sparkles className="w-3.5 h-3.5" />
  Interactive Role Selection
</div>
<h2 className="text-3xl sm:text-4xl font-extrabold tracking-tight text-white mb-4">
  Choose Your <span className="glow-text">NeuraMorphix Team Preferences</span>
</h2>
<p className="text-slate-300 text-base leading-relaxed">
  Selecting your <strong>1st Choice Role is compulsory</strong>. A <strong>2nd Choice Role is optional</strong>. Explore
all 10 specialized teams below.
</p>

{/* Counter Indicator Widget */}
<div className="mt-6 inline-flex items-center gap-3 px-6 py-2.5 rounded-2xl glass-panel border-slate-700/80">
  <span className="text-sm font-semibold text-slate-300">Role Status:</span>
  <div className="flex items-center gap-2">
    <span
      className={`px-3 py-1 rounded-lg text-xs font-extrabold font-mono transition-all ${
        firstChoice
          ? 'bg-emerald-500/20 text-emerald-300 border border-emerald-500/50'
          : 'bg-rose-500/20 text-rose-300 border border-rose-500/50'
      }`}
    >
      {firstChoice ? '1st Choice Selected (Ready to Apply)' : '1st Choice Required (Compulsory)'}
    </span>
    {secondChoice && (
      <span className="px-3 py-1 rounded-lg text-xs font-extrabold font-mono bg-cyan-500/20 text-cyan-300 border border-
cyan-500/50">
        + 2nd Choice Added (Optional)
      </span>
    )}
  </div>
</div>

{warningMsg && (
  <div className="mt-4 inline-flex items-center gap-2 px-4 py-2 rounded-xl bg-amber-950/80 border border-amber-500/40
text-amber-200 text-xs font-medium animate-fadeIn">
    <AlertCircle className="w-4 h-4 text-amber-400 shrink-0" />
    {warningMsg}
  </div>
)}
</div>

{/* Selected Preferences Summary Banner */}
<div className="mb-10 p-6 rounded-2xl glass-panel border-cyan-500/20 bg-slate-900/80 shadow-2xl relative overflow-hidden">
  <div className="absolute top-0 right-0 w-96 h-96 bg-cyan-500/5 rounded-full blur-3xl -z-10 pointer-events-none"></div>

  <div className="flex flex-col md:flex-row md:items-center justify-between gap-6">
    <div className="space-y-2">
      <h3 className="text-xs font-bold uppercase tracking-wider text-cyan-400 flex items-center gap-2">
        <CheckCircle2 className="w-4 h-4" />
        Your Selected Roles
      </h3>
      <div className="flex flex-col sm:flex-row sm:items-center gap-4 pt-1">
        <div className="flex items-center gap-2.5 px-4 py-2 rounded-xl bg-slate-950/80 border border-cyan-500/40">
          <span className="text-xl">👉</span>
          <div>
            <div className="text-[10px] text-cyan-400 uppercase font-bold">First Choice (Compulsory) *</div>
            <div className="text-sm font-bold text-cyan-300">
              {firstChoice || <span className="text-rose-400 italic">Select compulsory 1st role below...</span>}
            </div>
          </div>
        </div>

        <div className="flex items-center gap-2.5 px-4 py-2 rounded-xl bg-slate-950/80 border border-slate-800">
          <span className="text-xl">👉</span>
          <div>
            <div className="text-[10px] text-slate-400 uppercase font-semibold">Second Choice (Optional)</div>
            <div className="text-sm font-bold text-amber-300">
              {secondChoice || <span className="text-slate-500 italic">Optional (Select 2nd preference)...</span>}
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>

```

```

    { /* Buttons */}
    <div className="flex items-center gap-3 shrink-0">
      {(firstChoice || secondChoice) && (
        <button
          type="button"
          onClick={onClearPreferences}
          className="px-4 py-2.5 rounded-xl text-xs font-semibold bg-slate-800 hover:bg-slate-700 text-slate-300 border
border-slate-700 transition-all flex items-center gap-1.5 cursor-pointer"
        >
          <RefreshCw className="w-3.5 h-3.5" />
          Clear Selection
        </button>
      )}

      <button
        type="button"
        disabled={!firstChoice}
        onClick={onProceedToForm}
        className={`px-6 py-3 rounded-xl text-sm font-extrabold flex items-center gap-2 transition-all shadow-lg ${
          firstChoice
            ? 'bg-gradient-to-r from-cyan-500 to-blue-600 hover:from-cyan-400 hover:to-blue-500 text-slate-950 shadow-cyan-
500/25 ring-2 ring-cyan-300 cursor-pointer hover:scale-105'
            : 'bg-slate-800 text-slate-500 border border-slate-700 cursor-not-allowed opacity-60'
          }`}
      >
        Fill Application Details
        <ArrowRight className="w-4 h-4" />
      </button>
    </div>
  </div>
</div>

{ /* Grid of 10 Role Cards */}
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
  {roles.map((role) => (
    <RoleCard
      key={role.role_id}
      role={role}
      firstChoice={firstChoice}
      secondChoice={secondChoice}
      onSelectFirstChoice={handleSelectFirst}
      onSelectSecondChoice={handleSelectSecond}
    />
  ))}
</div>

{ /* ===== BOTTOM CTA – shown after selecting 1st choice ===== */}
{firstChoice && (
  <div className="mt-12 animate-fadeIn">
    { /* Summary recap */}
    <div className="max-w-2xl mx-auto p-6 rounded-2xl bg-gradient-to-br from-slate-900 via-slate-900 to-cyan-950/40 border
border-cyan-500/30 shadow-2xl shadow-cyan-500/10 text-center space-y-5">
      <div className="flex justify-center">
        <span className="px-4 py-1.5 rounded-full bg-emerald-950 text-emerald-300 text-xs font-bold uppercase border
border-emerald-500/30 flex items-center gap-1.5">
          <CheckCircle2 className="w-3.5 h-3.5" />
          Role Selection Done
        </span>
      </div>
    </div>

    <h3 className="text-xl sm:text-2xl font-extrabold text-white leading-tight">
      Ready to Apply!
    </h3>

    { /* Chosen roles recap */}
    <div className="flex flex-col sm:flex-row items-center justify-center gap-3 text-sm">
      <div className="flex items-center gap-2 px-4 py-2 rounded-xl bg-slate-950 border border-cyan-500/40">
        <span className="text-xl">👤</span>
        <div className="text-left">
          <div className="text-[10px] text-cyan-400 font-bold uppercase">1st Choice</div>
          <div className="text-cyan-300 font-bold text-xs">{firstChoice}</div>
        </div>
      </div>
      {secondChoice ? (
        <div className="flex items-center gap-2 px-4 py-2 rounded-xl bg-slate-950 border border-amber-500/40">
          <span className="text-xl">👤</span>

```

```

        <div className="text-left">
          <div className="text-[10px] text-amber-400 font-bold uppercase">2nd Choice</div>
          <div className="text-amber-300 font-bold text-xs">{secondChoice}</div>
        </div>
      </div>
    ) : (
      <div className="px-4 py-2 rounded-xl bg-slate-950/60 border border-slate-700 text-slate-500 text-xs italic">
        No 2nd choice (optional)
      </div>
    )
  </div>

  <p className="text-slate-400 text-xs max-w-md mx-auto">
    Click below to fill in your personal information, skills, and experience to complete your NeuraMorphix 2026
    recruitment application.
  </p>

  { /* Primary CTA button */ }
  <button
    type="button"
    onClick={onProceedToForm}
    className="w-full sm:w-auto mx-auto px-10 py-4 rounded-2xl bg-gradient-to-r from-cyan-500 to-blue-600 hover:from-
cyan-400 hover:to-blue-500 text-slate-950 font-extrabold text-base flex items-center justify-center gap-2 shadow-
[0_0_30px_rgba(56,189,248,0.35)] ring-2 ring-cyan-300 transition-all hover:scale-105 cursor-pointer"
  >
    Fill Application Details
    <ArrowRight className="w-5 h-5" />
  </button>

  <button
    type="button"
    onClick={onClearPreferences}
    className="text-xs text-slate-500 hover:text-slate-300 underline transition-colors"
  >
    Reset selection and start over
  </button>
</div>
</div>
)}

{ /* ===== MOBILE STICKY BOTTOM BAR – appears after 1st choice selected ===== */ }
{firstChoice && (
  <div className="fixed bottom-0 left-0 right-0 z-50 md:hidden bg-slate-950/95 backdrop-blur-md border-t border-cyan-500/30
px-4 py-3 shadow-[0_-4px_24px_rgba(6,182,212,0.15)]">
    <div className="flex items-center gap-3 max-w-lg mx-auto">
      <div className="flex-1 min-w-0">
        <div className="text-[10px] text-cyan-400 font-bold uppercase">Selected:</div>
        <div className="text-xs font-bold text-white truncate">{firstChoice}</div>
        {secondChoice && (
          <div className="text-[10px] text-amber-300 truncate">+ {secondChoice}</div>
        )}
      </div>
      <button
        type="button"
        onClick={onProceedToForm}
        className="shrink-0 px-5 py-3 rounded-xl bg-gradient-to-r from-cyan-500 to-blue-600 hover:from-cyan-400 hover:to-
blue-500 text-slate-950 font-extrabold text-xs flex items-center gap-1.5 shadow-lg cursor-pointer transition-all active:scale-95"
      >
        Fill Application
        <ArrowRight className="w-4 h-4" />
      </button>
    </div>
  </div>
)}

{ /* Bottom padding to avoid mobile sticky bar covering content */ }
{firstChoice && <div className="md:hidden h-20" />}
</section>
);
};

```

Applicant status search bar and dynamic timeline showing progress from submission to acceptance.

```
import React, { useState, useEffect } from 'react';
import type { Applicant, ApplicationStatus } from '../types/recruitment';
import { DatabaseService } from '../services/db';
import {
  Search,
  CheckCircle2,
  Clock,
  Send,
  AlertCircle,
  FileQuestion,
  UserCheck,
  UserX,
  Award,
  Sparkles,
} from 'lucide-react';

interface StatusTrackerProps {
  initialAppId?: string | null;
}

export const StatusTracker: React.FC<StatusTrackerProps> = ({ initialAppId }) => {
  const [appIdInput, setAppIdInput] = useState(initialAppId || '');
  const [searchedApplicant, setSearchedApplicant] = useState<Applicant | null>(() => {
    return initialAppId ? DatabaseService.getApplicantById(initialAppId) || null : null;
  });
  const [notFound, setNotFound] = useState(false);

  // Response field for Information Requested status
  const [infoReplyInput, setInfoReplyInput] = useState('');
  const [isSubmittingReply, setIsSubmittingReply] = useState(false);
  const [replySuccessMsg, setReplySuccessMsg] = useState<string | null>(null);

  useEffect(() => {
    if (initialAppId) {
      const found = DatabaseService.getApplicantById(initialAppId);
      if (found) {
        setAppIdInput(initialAppId);
        setSearchedApplicant(found);
        setNotFound(false);
      }
    }
  }, [initialAppId]);

  const handleSearch = (e: React.FormEvent) => {
    e.preventDefault();
    setNotFound(false);
    setReplySuccessMsg(null);

    const found = DatabaseService.getApplicantById(appIdInput);
    if (found) {
      setSearchedApplicant(found);
    } else {
      setSearchedApplicant(null);
      setNotFound(true);
    }
  };

  const handleInfoReplySubmit = (e: React.FormEvent) => {
    e.preventDefault();
    if (!searchedApplicant || !infoReplyInput.trim()) return;

    setIsSubmittingReply(true);

    const updated = DatabaseService.updateApplicant(searchedApplicant.id, {
      requested_info_response: infoReplyInput.trim(),
      status: 'Information Received',
    });

    if (updated) {
      setSearchedApplicant(updated);
    }
  };
};
```



```

        className="w-full sm:w-auto px-8 py-3 rounded-xl text-sm font-bold bg-cyan-500 hover:bg-cyan-400 text-slate-950 flex
items-center justify-center gap-2 shadow-lg shadow-cyan-500/20 cursor-pointer shrink-0"
    >
        <Search className="w-4 h-4" />
        CHECK APPLICATION STATUS
    </button>
</form>
</div>

{notFound && (
    <div className="p-6 rounded-2xl glass-panel border-rose-500/30 text-center space-y-2 animate-fadeIn">
        <AlertCircle className="w-8 h-8 text-rose-400 mx-auto mb-2" />
        <h3 className="text-lg font-bold text-white">Application Not Found</h3>
        <p className="text-sm text-slate-400 max-w-md mx-auto">
            No recruitment record matches Application ID <span className="font-mono text-white">{appIdInput}</span>. Please
verify your Application ID.
        </p>
    </div>
)}

{/* APPLICANT DETAILS & TIMELINE */}
{searchedApplicant && (
    <div className="glass-panel p-6 sm:p-8 rounded-2xl space-y-8 animate-fadeIn">
        {/* Top Bar with ID and Status Badge */}
        <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4 pb-6 border-b border-slate-800">
            <div>
                <div className="text-xs text-slate-400 uppercase font-semibold">Application ID</div>
                <h3 className="text-2xl font-black font-mono text-cyan-300">{searchedApplicant.application_id}</h3>
                <p className="text-xs text-slate-400 mt-1">Applicant: <strong className="text-white">{searchedApplicant.full_name}</strong> {searchedApplicant.college}</p>
            </div>

            <div className="flex items-center gap-2">
                <span
                    className={`px-4 py-2 rounded-xl text-xs font-bold flex items-center gap-2 border shadow-lg ${
                        searchedApplicant.status === 'Accepted'
                            ? 'bg-emerald-500/20 text-emerald-300 border-emerald-500/50'
                            : searchedApplicant.status === 'Declined'
                            ? 'bg-rose-500/20 text-rose-300 border-rose-500/50'
                            : searchedApplicant.status === 'Interview'
                            ? 'bg-amber-500/20 text-amber-300 border-amber-500/50'
                            : searchedApplicant.status === 'Information Requested'
                            ? 'bg-purple-500/20 text-purple-300 border-purple-500/50'
                            : 'bg-cyan-500/20 text-cyan-300 border-cyan-500/50'
                    }`}>
                </span>
                {searchedApplicant.status === 'Accepted' && <UserCheck className="w-4 h-4" />}
                {searchedApplicant.status === 'Declined' && <UserX className="w-4 h-4" />}
                {searchedApplicant.status === 'Information Requested' && <FileQuestion className="w-4 h-4" />}
                Status: {searchedApplicant.status}
            </span>
        </div>
    </div>

    {/* VISUAL PROGRESS TIMELINE */}
    <div>
        <h4 className="text-xs font-bold uppercase tracking-wider text-slate-400 mb-6">Visual Application Timeline</h4>
        {searchedApplicant.status === 'Declined' ? (
            <div className="p-4 rounded-xl bg-rose-950/60 border border-rose-500/40 text-rose-200 text-sm flex items-center
gap-3">
                <UserX className="w-6 h-6 text-rose-400 shrink-0" />
                <div>
                    <div className="font-bold">Application Status: Declined</div>
                    <div className="text-xs text-rose-300 mt-0.5">
                        Thank you for applying for the NeuraMorphix 2026 cycle. Unfortunately, your application was not selected for
this recruitment period.
                    </div>
                </div>
            </div>
        ) : (
            <div className="relative py-4">
                <div className="absolute top-1/2 left-0 right-0 h-1 bg-slate-800 -translate-y-1/2 rounded-full -z-0"></div>
                <div
                    className="absolute top-1/2 left-0 h-1 bg-gradient-to-r from-cyan-500 to-emerald-400 -translate-y-1/2 rounded-
full transition-all duration-500 -z-0"
                    style={{

```

```

        width: `${(Math.max(0, currentStepIdx) / (TIMELINE_STEPS.length - 1)) * 100}%`,
      }}
    ></div>

    <div className="grid grid-cols-5 gap-2 relative z-10 text-center">
      {TIMELINE_STEPS.map((stepItem, idx) => {
        const isCompleted = idx <= currentStepIdx;
        const isCurrent = idx === currentStepIdx;
        return (
          <div key={idx} className="flex flex-col items-center">
            <div
              className={`w-9 h-9 rounded-full flex items-center justify-center font-bold text-xs transition-all ${
                isCompleted
                  ? 'bg-cyan-500 text-slate-950 ring-4 ring-cyan-500/20 shadow-lg shadow-cyan-500/30'
                  : 'bg-slate-800 text-slate-500 border border-slate-700'
              }`}
            >
              {isCompleted ? <CheckCircle2 className="w-5 h-5 stroke-[3]" /> : idx + 1}
            </div>
            <span
              className={`text-[11px] font-semibold mt-2.5 ${
                isCurrent ? 'text-cyan-300 font-extrabold' : isCompleted ? 'text-slate-200' : 'text-slate-500'
              }`}
            >
              {stepItem.label}
            </span>
          </div>
        );
      })}
    </div>
  </div>
)}

/* ACCEPTED FINAL ASSIGNED TEAM BANNER */
{searchedApplicant.status === 'Accepted' && (
  <div className="p-6 rounded-2xl bg-gradient-to-r from-emerald-950/80 via-slate-900 to-cyan-950/80 border border-emerald-500/50 shadow-2xl flex items-center gap-4">
    <div className="p-3.5 rounded-2xl bg-emerald-500/20 text-emerald-300 border border-emerald-500/40">
      <Award className="w-8 h-8" />
    </div>
    <div>
      <div className="text-xs font-bold text-emerald-400 uppercase tracking-wider">Accepted into Team</div>
      <div className="text-xl font-black text-white">
        {searchedApplicant.final_assigned_team || searchedApplicant.first_preference}
      </div>
      <p className="text-xs text-slate-300 mt-1">
        Congratulations! Onboarding details will be communicated via email shortly.
      </p>
    </div>
  </div>
)}

/* INFORMATION REQUESTED ACTION BOX */
{((searchedApplicant.status === 'Information Requested' || searchedApplicant.requested_info_question) && (
  <div className="p-6 rounded-2xl bg-purple-950/80 border border-purple-500/50 space-y-4">
    <div className="flex items-center gap-3">
      <FileQuestion className="w-6 h-6 text-purple-400" />
      <div>
        <h4 className="text-sm font-bold text-white">Additional Information Requested by Recruiter</h4>
        <p className="text-xs text-purple-200 mt-0.5">
          {searchedApplicant.requested_info_question}
        </p>
      </div>
    </div>
    <div>
      {searchedApplicant.requested_info_response ? (
        <div className="p-4 rounded-xl bg-slate-950/90 border border-slate-800 text-xs">
          <span className="text-slate-400 block font-semibold mb-1">Your Submitted Response:</span>
          <p className="text-slate-200 whitespace-pre-wrap">{searchedApplicant.requested_info_response}</p>
          <span className="inline-block mt-2 text-[10px] text-emerald-400 font-bold uppercase">✓ Status: Information
Received</span>
        </div>
      ) : (
        <form onSubmit={handleInfoReplySubmit} className="space-y-3">
          <textarea

```



```

        required
        rows={3}
        placeholder="Type your response here (e.g. GitHub link, portfolio details, or clarifications)..."
        value={infoReplyInput}
        onChange={(e) => setInfoReplyInput(e.target.value)}
        className="w-full p-3 rounded-xl glass-input text-xs"
      />
      <button
        type="submit"
        disabled={isSubmittingReply || !infoReplyInput.trim()}
        className="px-5 py-2.5 rounded-xl bg-purple-500 hover:bg-purple-400 text-slate-950 text-xs font-bold flex
items-center gap-2 shadow-lg"
      >
        <Send className="w-3.5 h-3.5" />
        Submit Requested Information
      </button>
    </form>
  )}

  {replySuccessMsg && (
    <div className="p-3 rounded-xl bg-emerald-950 text-emerald-300 text-xs font-medium">
      {replySuccessMsg}
    </div>
  )}
</div>
))

{/* INTERVIEW DETAILS BOX */}
{searchedApplicant.interview_details && (
  <div className="p-5 rounded-xl bg-amber-950/60 border border-amber-500/40 text-amber-200 text-xs space-y-1">
    <div className="font-bold flex items-center gap-2 text-amber-300">
      <Sparkles className="w-4 h-4" />
      Interview Information
    </div>
    <p className="text-slate-300 whitespace-pre-wrap leading-relaxed">{searchedApplicant.interview_details}</p>
  </div>
)}

{/* Application Summary Breakdown */}
<div className="grid grid-cols-1 sm:grid-cols-2 gap-4 text-xs pt-4 border-t border-slate-800">
  <div className="p-5 rounded-xl bg-slate-900 border border-slate-800">
    <span className="text-slate-400 block font-semibold mb-1">🔖 First Preference</span>
    <span className="text-sm font-bold text-cyan-300">{searchedApplicant.first_preference}</span>
  </div>

  <div className="p-4 rounded-xl bg-slate-900 border border-slate-800">
    <span className="text-slate-400 block font-semibold mb-1">🔖 Second Preference</span>
    <span className="text-sm font-bold text-amber-300">{searchedApplicant.second_preference}</span>
  </div>
</div>
</div>
)}
</div>
);
};

```

Live Email Inbox Drawer Component

<src/components/EmailInboxDrawer.tsx>

Floating drawer displaying live previews of sent emails in real-time.

```

import React, { useState, useEffect } from 'react';
import type { EmailLog } from '../types/recruitment';
import { DatabaseService } from '../services/db';
import { Mail, X, CheckCircle2, ChevronRight } from 'lucide-react';

export const EmailInboxDrawer: React.FC = () => {
  const [isOpen, setIsOpen] = useState(false);
  const [emailLogs, setEmailLogs] = useState<EmailLog[]>(() => DatabaseService.getEmailLogs());
  const [selectedEmail, setSelectedEmail] = useState<EmailLog | null>(null);
  const [newUnreadCount, setNewUnreadCount] = useState(0);

  const fetchLogs = () => {

```

```

const logs = DatabaseService.getEmailLogs();
setEmailLogs(logs);
};

useEffect(() => {
  const handleEmailSent = (e: Event) => {
    fetchLogs();
    setNewUnreadCount((prev) => prev + 1);
    const customEvent = e as CustomEvent<EmailLog>;
    if (customEvent.detail) {
      setSelectedEmail(customEvent.detail);
    }
  };
};

window.addEventListener('neuramorphix_email_sent', handleEmailSent);
return () => {
  window.removeEventListener('neuramorphix_email_sent', handleEmailSent);
};
}, []);

const toggleOpen = () => {
  setIsOpen(!isOpen);
  if (!isOpen) {
    setNewUnreadCount(0);
    fetchLogs();
  }
};

return (
  <>
    { /* Floating Action Button */ }
    <button
      type="button"
      onClick={toggleOpen}
      className="fixed bottom-6 left-6 z-40 px-4 py-3 rounded-2xl glass-panel bg-slate-900/90 text-cyan-300 hover:text-white
border-cyan-500/40 shadow-2xl flex items-center gap-2.5 transition-all hover:scale-105"
    >
      <div className="relative">
        <Mail className="w-5 h-5" />
        {newUnreadCount > 0 && (
          <span className="absolute -top-1.5 -right-1.5 w-4 h-4 rounded-full bg-rose-500 text-white text-[10px] font-bold flex
items-center justify-center">
            {newUnreadCount}
          </span>
        )}
      </div>
      <span className="text-xs font-bold">Simulated Sent Emails ({emailLogs.length})</span>
    </button>

    { /* Drawer Overlay */ }
    {isOpen && (
      <div className="fixed inset-0 z-50 bg-slate-950/80 backdrop-blur-md flex justify-end">
        <div className="w-full max-w-xl bg-slate-900 border-l border-slate-800 h-full flex flex-col shadow-2xl animate-
slideLeft">
          { /* Drawer Header */ }
          <div className="p-6 border-b border-slate-800 flex items-center justify-between">
            <div className="flex items-center gap-2">
              <Mail className="w-5 h-5 text-cyan-400" />
              <h3 className="text-lg font-bold text-white">Sent Email Notification Logs</h3>
            </div>
            <button
              type="button"
              onClick={() => setIsOpen(false)}
              className="p-2 rounded-lg text-slate-400 hover:text-white"
            >
              <X className="w-5 h-5" />
            </button>
          </div>

          { /* Content Split: Log List vs Email Preview */ }
          <div className="flex-1 overflow-y-auto p-4 space-y-4">
            {selectedEmail ? (
              <div className="space-y-4 animate-fadeIn">
                <button
                  type="button"
                  onClick={() => setSelectedEmail(null)}

```

```

        className="text-xs text-cyan-400 hover:underline flex items-center gap-1 font-semibold"
      >
      ← Back to all sent emails
    </button>

    <div className="p-6 rounded-2xl glass-panel border-cyan-500/30 space-y-4">
      <div className="border-b border-slate-800 pb-3">
        <div className="text-[10px] text-cyan-400 font-bold uppercase">To: {selectedEmail.recipient_email}</div>
        <h4 className="text-base font-extrabold text-white mt-1">{selectedEmail.subject}</h4>
        <div className="text-[11px] text-slate-400 mt-1 flex justify-between">
          <span>App ID: {selectedEmail.application_id}</span>
          <span>{new Date(selectedEmail.sent_at).toLocaleString()}</span>
        </div>
      </div>

      <div className="text-xs text-slate-200 whitespace-pre-wrap font-mono leading-relaxed bg-slate-950 p-4 rounded-xl border border-slate-850">
        {selectedEmail.body_html}
      </div>

      <div className="text-[10px] text-emerald-400 font-bold uppercase flex items-center gap-1 pt-2">
        <CheckCircle2 className="w-3.5 h-3.5" /> Sent automatically via NeuraMorphix Email Engine
      </div>
    </div>
  </div>
) : emailLogs.length === 0 ? (
  <div className="text-center py-16 text-slate-500 text-xs">
    No automated emails sent yet. Submit an application or trigger admin actions to view generated emails!
  </div>
) : (
  <div className="space-y-2">
    {emailLogs.map((log) => (
      <div
        key={log.email_id}
        onClick={() => setSelectedEmail(log)}
        className="p-4 rounded-xl bg-slate-950/80 hover:bg-slate-800/80 border border-slate-800 transition-all cursor-pointer flex items-center justify-between"
      >
        <div className="space-y-1">
          <div className="flex items-center gap-2">
            <span className="text-xs font-bold text-white">{log.subject}</span>
            <span className="px-2 py-0.5 rounded bg-cyan-950 text-cyan-300 text-[9px] font-bold uppercase">
              {log.email_type}
            </span>
          </div>
          <div className="text-[11px] text-slate-400">
            To: {log.recipient_email} ({log.application_id})
          </div>
          <div>
            <ChevronRight className="w-4 h-4 text-slate-500" />
          </div>
        </div>
      </div>
    ))}
  </div>
)
</div>
</div>
</div>
</div>
);
};

```

Header Navigation Component

<src/components/Header.tsx>

Top navbar with logo, recruitment countdown timer, and portal status badge.

```

import React from 'react';
import { DatabaseService } from '../services/db';
import { NeuraMorphixLogo } from './NeuraMorphixLogo';
import { Search, UserPlus } from 'lucide-react';

interface HeaderProps {

```

```

currentTab: 'home' | 'apply' | 'track' | 'admin';
onSelectTab: (tab: 'home' | 'apply' | 'track' | 'admin') => void;
}

export const Header: React.FC<HeaderProps> = ({ currentTab, onSelectTab }) => {
  const windowStatus = DatabaseService.isRecruitmentOpen();
  const [timeStr, setTimeStr] = React.useState('');

  React.useEffect(() => {
    const updateTime = () => {
      const now = new Date();
      setTimeStr(
        now.toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', second: '2-digit' }) + ' IST'
      );
    };
    updateTime();
    const interval = setInterval(updateTime, 1000);
    return () => clearInterval(interval);
  }, []);

  return (
    <header className="sticky top-0 z-40 w-full glass-panel border-b border-slate-800/80 bg-slate-950/80">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 h-16 sm:h-20 flex items-center justify-between gap-2">
        {/* Brand Logo */}
        <div
          onClick={() => onSelectTab('home')}
          className="flex items-center gap-2 sm:gap-3 cursor-pointer group shrink-0"
        >
          <NeuraMorphixLogo size={36} />
          <div>
            <span className="text-base sm:text-xl font-extrabold tracking-wider text-white group-hover:text-cyan-300 transition-colors">
              NEURAMORPHIX
            </span>
            {/* Full subtitle – desktop only */}
            <div className="hidden sm:flex items-center gap-2 text-[10px] text-slate-400 font-semibold uppercase tracking-widest">
              <span>Recruitment 2026</span>
              <span className="text-cyan-400">•</span>
              <span className="text-slate-300">05 Sep – 18 Sep 2026</span>
              {timeStr} && (
                <div>
                  <span className="text-cyan-400">•</span>
                  <span className="text-cyan-300 font-mono font-bold">{timeStr}</span>
                </div>
              )
            </div>
            {/* Compact – mobile only */}
            <div className="flex sm:hidden text-[9px] text-slate-400 font-semibold uppercase tracking-wide">
              Recruitment 2026
            </div>
          </div>
        </div>

        {/* Navigation Tabs – desktop only */}
        <nav className="hidden md:flex items-center gap-1.5 bg-slate-900/90 p-1.5 rounded-2xl border border-slate-800">
          <button
            type="button"
            onClick={() => onSelectTab('home')}
            className={`px-4 py-2 rounded-xl text-xs font-bold transition-all ${
              currentTab === 'home'
                ? 'bg-cyan-500 text-slate-950 shadow-md'
                : 'text-slate-300 hover:text-white'
            }`}
          >
            Explore Teams
          </button>

          <button
            type="button"
            onClick={() => onSelectTab('apply')}
            className={`px-4 py-2 rounded-xl text-xs font-bold transition-all flex items-center gap-1.5 ${
              currentTab === 'apply'
                ? 'bg-cyan-500 text-slate-950 shadow-md'
                : 'text-slate-300 hover:text-white'
            }`}
          >

```

```

    <UserPlus className="w-3.5 h-3.5" />
    Apply Now
  </button>

  <button
    type="button"
    onClick={() => onSelectTab('track')}
    className={`px-4 py-2 rounded-xl text-xs font-bold transition-all flex items-center gap-1.5 ${
      currentTab === 'track'
        ? 'bg-cyan-500 text-slate-950 shadow-md'
        : 'text-slate-300 hover:text-white'
    }`}
  >
    <Search className="w-3.5 h-3.5" />
    Track Status
  </button>
</nav>

{/* Recruitment Status Pill */}
<div className="flex items-center shrink-0">
  <span
    className={`px-2.5 sm:px-3.5 py-1.5 rounded-full text-[10px] sm:text-xs font-extrabold flex items-center gap-1.5 border shadow-inner ${
      windowStatus.isOpen
        ? 'bg-emerald-950/80 text-emerald-300 border-emerald-500/40'
        : 'bg-rose-950/80 text-rose-300 border-rose-500/40'
    }`}
  >
    <span
      className={`w-2 h-2 rounded-full shrink-0 ${
        windowStatus.isOpen ? 'bg-emerald-400 shadow-[0_0_8px_rgba(52,211,153,0.8)]' : 'bg-rose-400'
      }`}
    ></span>
    <span className="hidden sm:inline">{windowStatus.isOpen ? 'RECRUITMENT OPEN' : 'RECRUITMENT CLOSED'}</span>
    <span className="sm:hidden">{windowStatus.isOpen ? 'OPEN' : 'CLOSED'}</span>
  </span>
</div>
</div>

{/* Mobile bottom nav tabs - only visible on mobile */}
<div className="flex md:hidden border-t border-slate-800 bg-slate-950/95">
  <button
    type="button"
    onClick={() => onSelectTab('home')}
    className={`flex-1 py-2.5 text-[11px] font-bold transition-all flex flex-col items-center gap-0.5 ${
      currentTab === 'home' ? 'text-cyan-400 border-b-2 border-cyan-400' : 'text-slate-500'
    }`}
  >
    <span>🏠</span>
    <span>Teams</span>
  </button>
  <button
    type="button"
    onClick={() => onSelectTab('apply')}
    className={`flex-1 py-2.5 text-[11px] font-bold transition-all flex flex-col items-center gap-0.5 ${
      currentTab === 'apply' ? 'text-cyan-400 border-b-2 border-cyan-400' : 'text-slate-500'
    }`}
  >
    <UserPlus className="w-4 h-4" />
    <span>Apply</span>
  </button>
  <button
    type="button"
    onClick={() => onSelectTab('track')}
    className={`flex-1 py-2.5 text-[11px] font-bold transition-all flex flex-col items-center gap-0.5 ${
      currentTab === 'track' ? 'text-cyan-400 border-b-2 border-cyan-400' : 'text-slate-500'
    }`}
  >
    <Search className="w-4 h-4" />
    <span>Track</span>
  </button>
</div>
</header>

```

```
);  
};
```

Footer Component

[src/components/Footer.tsx](#)

Footer containing NeuraMorphix links, admin portal lock button, and copyright info.

```
import React from 'react';  
import { NeuraMorphixLogo } from './NeuraMorphixLogo';  
  
export const Footer: React.FC = () => {  
  return (  
    <footer className="w-full bg-slate-950 border-t border-slate-800/80 py-12 px-4 sm:px-6 lg:px-8 mt-auto">  
      <div className="max-w-7xl mx-auto grid grid-cols-1 md:grid-cols-4 gap-8">  
        <div className="space-y-3">  
          <div className="flex items-center gap-2">  
            <NeuraMorphixLogo size={28} />  
            <span className="text-lg font-black tracking-wider text-white">NEURAMORPHIX</span>  
          </div>  
          <p className="text-xs text-slate-400 leading-relaxed">  
            Building the next generation of artificial intelligence, intelligent systems, hardware integration, and full-stack  
software products.  
          </p>  
        </div>  
  
        <div>  
          <h4 className="text-xs font-bold text-slate-200 uppercase tracking-wider mb-3">Recruitment Period</h4>  
          <ul className="space-y-2 text-xs text-slate-400">  
            <li>Opening Date: <strong className="text-cyan-300">05 September 2026</strong></li>  
            <li>Closing Deadline: <strong className="text-rose-300">18 September 2026</strong></li>  
            <li>Cycle: <strong className="text-white">Annual Team Recruitment 2026</strong></li>  
          </ul>  
        </div>  
  
        <div>  
          <h4 className="text-xs font-bold text-slate-200 uppercase tracking-wider mb-3">Contact & Support</h4>  
          <ul className="space-y-2 text-xs text-slate-400">  
            <li>Email: <a href="mailto:recruitment@neuramorphix.org" className="text-cyan-400  
hover:underline">recruitment@neuramorphix.org</a></li>  
            <li>Web: <span className="text-slate-300">neuramorphix.org</span></li>  
            <li>Location: Innovation Hub</li>  
          </ul>  
        </div>  
  
        <div>  
          <h4 className="text-xs font-bold text-slate-200 uppercase tracking-wider mb-3">System Features</h4>  
          <div className="flex flex-wrap gap-1.5 text-[11px] text-slate-400">  
            <span className="px-2 py-1 rounded bg-slate-900 border border-slate-800">Role Preference Engine</span>  
            <span className="px-2 py-1 rounded bg-slate-900 border border-slate-800">Auto Email Dispatch</span>  
            <span className="px-2 py-1 rounded bg-slate-900 border border-slate-800">Recruitment Platform</span>  
            <span className="px-2 py-1 rounded bg-slate-900 border border-slate-800">Status Progress Stepper</span>  
          </div>  
        </div>  
      </div>  
  
      <div className="max-w-7xl mx-auto border-t border-slate-900 mt-8 pt-6 flex flex-col sm:flex-row items-center justify-between text-xs text-slate-500">  
        <div>© 2026 NeuraMorphix. All rights reserved.</div>  
        <div className="mt-2 sm:mt-0 font-medium">NeuraMorphix Team Recruitment Platform</div>  
      </div>  
    </footer>  
  );  
};
```

Custom SVG neural brain icon and branded typography component.

```
import React from 'react';

interface NeuraMorphixLogoProps {
  className?: string;
  size?: number;
}

export const NeuraMorphixLogo: React.FC<NeuraMorphixLogoProps> = ({
  className = 'w-10 h-10',
  size = 40,
}) => {
  return (
    <svg
      width={size}
      height={size}
      viewBox="0 0 100 100"
      fill="none"
      xmlns="http://www.w3.org/2000/svg"
      className={` ${className} drop-shadow-[0_0_12px_rgba(56,189,248,0.5)] transition-transform duration-300 hover:scale-105`}
    >
      <defs>
        { /* Main Ribbon Gradient */}
        <linearGradient id="nmx-infinity-grad" x1="0%" y1="0%" x2="100%" y2="100%">
          <stop offset="0%" stopColor="#c084fc" />
          <stop offset="35%" stopColor="#a855f7" />
          <stop offset="65%" stopColor="#3b82f6" />
          <stop offset="100%" stopColor="#38bdf8" />
        </linearGradient>

        { /* Glow Filter */}
        <filter id="nmx-glow" x="-20%" y="-20%" width="140%" height="140%">
          <feGaussianBlur stdDeviation="3" result="blur" />
          <feComposite in="SourceGraphic" in2="blur" operator="over" />
        </filter>
      </defs>

      { /* Futuristic 3D Infinity Ribbon Path */}
      <g filter="url(#nmx-glow)">
        { /* Back Loop Shade */}
        <path
          d="M 30,50 C 15,30 15,70 30,70 C 45,70 55,30 70,30 C 85,30 85,70 70,70 C 55,70 45,30 30,30 C 15,30 15,70 30,50 Z"
          fill="none"
          stroke="url(#nmx-infinity-grad)"
          strokeWidth="14"
          strokeLinecap="round"
          strokeLinejoin="round"
        />

        { /* Inner Highlight Loop */}
        <path
          d="M 30,50 C 15,32 15,68 30,68 C 45,68 55,32 70,32 C 85,32 85,68 70,68 C 55,68 45,32 30,32 C 15,32 15,68 30,50 Z"
          fill="none"
          stroke="ffffff"
          strokeWidth="2.5"
          strokeOpacity="0.4"
          strokeLinecap="round"
        />
      </g>
    </svg>
  );
};
```

Icon resolver mapping team categories to Lucide icons.

```
import React from 'react';
import type { LucideProps } from 'lucide-react';
import {
  Cpu,
  Code2,
  Palette,
  Megaphone,
  Layers,
  Radio,
  CalendarCheck,
  Users,
  FileText,
  Lightbulb,
} from 'lucide-react';

interface TeamIconProps extends LucideProps {
  name: string;
}

export const TeamIcon: React.FC<TeamIconProps> = ({ name, className = 'w-6 h-6', ...props }) => {
  switch (name) {
    case 'Cpu':
      return <Cpu className={className} {...props} />;
    case 'Code2':
      return <Code2 className={className} {...props} />;
    case 'Palette':
      return <Palette className={className} {...props} />;
    case 'Megaphone':
      return <Megaphone className={className} {...props} />;
    case 'Layers':
      return <Layers className={className} {...props} />;
    case 'Radio':
      return <Radio className={className} {...props} />;
    case 'CalendarCheck':
      return <CalendarCheck className={className} {...props} />;
    case 'Users':
      return <Users className={className} {...props} />;
    case 'FileText':
      return <FileText className={className} {...props} />;
    case 'Lightbulb':
      return <Lightbulb className={className} {...props} />;
    default:
      return <Cpu className={className} {...props} />;
  }
};
```